

ĐẠI HỌC QUỐC GIA THÀNH PHỐ HỒ CHÍ MINH
TRƯỜNG ĐẠI HỌC BÁCH KHOA
KHOA KHOA HỌC VÀ KỸ THUẬT MÁY TÍNH



Đồ án tổng hợp - CNPM (CO3103)

Báo cáo

Ứng dụng nghe nhạc trực tuyến *MusicHUB*

GVHD: ThS. Trần Trương Tuấn Phát

Sinh viên: Lữ Chấn Vũ - 2313955 (*Nhóm 10, Leader*)
Nguyễn Phú Vinh - 2313922 (*Nhóm 10*)
Trần Dương Khiết Nhi - 2312509 (*Nhóm 10*)
Lê Minh Khoa - 2311593 (*Nhóm 10*)
Lê Minh Trí - 2313593 (*Nhóm 10*)

TP. HỒ CHÍ MINH, 09/2025

Mục lục

Danh sách hình ảnh	4
Danh sách bảng biểu	4
Danh sách thành viên và nhiệm vụ	4
1 Tổng quan về đề tài	5
1.1 Mô tả đề tài	5
1.2 Mục tiêu đề tài	5
1.3 Phạm vi đề tài	5
2 Functional Requirements - Non-functional Requirements	6
2.1 Functional Requirements	6
2.1.1 Functional Requirements	6
2.1.1.a Quản lý tài khoản và Xác thực (Authentication & Account Management)	6
2.1.1.b Trình phát nhạc và Điều khiển (Music Player & Controls)	6
2.1.1.c Quản lý danh sách nhạc (Playlist Management)	7
2.1.1.d Khám phá và Tìm kiếm (Discovery & Search)	7
2.1.1.e Tương tác xã hội (Social Interaction)	7
2.1.1.f Chức năng dành cho Nghệ sĩ và Quản trị (Artist & Admin Features)	7
2.1.2 Non-interactive Requirements	8
2.1.2.a Tính năng Streaming ở nhiều mức bitrate	8
2.1.2.b Tính năng gợi ý bài hát dựa vào lượt nghe gần đây	8
2.1.2.c Tính năng tự động phát bài hát tiếp theo trong playlist/queue	9
2.1.2.d Tính năng thông báo khi Nghệ sĩ được follow phát hành bài hát/album mới	10
2.1.2.e Tính năng thống kê thời gian nghe và lượt nghe	10
2.1.2.f Tính năng tạo và đồng bộ lời bài hát (Lyric Sync)	11
2.2 Non-functional Requirements	12
2.2.1 Hiệu năng (Performance)	12
2.2.2 Bảo mật (Security)	12
2.2.3 Độ tin cậy (Reliability)	12
2.2.4 Khả năng sử dụng (Usability)	13
2.2.5 Khả năng mở rộng (Scalability)	13
2.2.6 Tính sẵn sàng (Availability)	13
2.2.7 Tính tương thích (Compatibility)	13
2.2.8 Tuân thủ pháp lý (Compliance)	14
2.2.9 Khả năng kiểm toán (Audit)	14
2.2.10 Hiệu quả sử dụng tài nguyên (Efficiency)	14
2.2.11 Khả năng bảo trì (Maintainability)	14
3 Use-case Diagram và Use-case Scenario	15
3.1 Use-case Diagram	15
3.2 Use-case Scenario	16
3.2.1 Use-case Upload bài hát/album	16
3.2.2 Use-case Đăng nhập & Đăng ký	17



3.2.3	Use-case Quản lý tài khoản	18
3.2.4	Use-case Tương tác	19
3.2.5	Use-case Nghe nhạc	20
3.2.6	Use-case Khám phá và tìm kiếm	21
3.2.7	Use-case Nhận diện bài hát	22
3.2.8	Use-case Quản lý danh sách nhạc	23
3.2.9	Use-case Quản trị hệ thống	24
4	Phân tích hệ thống	25
4.1	Class Diagram	25
4.2	Mô tả phương thức	26
4.2.1	Nhóm lớp Người dùng (User Group)	26
4.2.2	Nhóm lớp Nghiệp vụ và Dữ liệu (Business & Data)	26
4.2.3	Nhóm lớp Hệ thống và Dịch vụ (System & Services)	27
5	Thiết kế Cơ sở dữ liệu	27
5.1	User Schema	27
5.2	Track Schema	28
5.3	Playlist Schema	29
5.4	Notification Schema	30
5.5	Comment Schema	31
5.6	Follow Schema	32
6	Thiết kế hành vi và giao diện người dùng	34
6.1	Đăng nhập - đăng kí	34
6.1.1	Đăng nhập - Login	34
6.1.2	Đăng ký - Register	37
6.2	Quản lý tài khoản - Account Management	40
6.3	Quản lý danh sách nhạc - Playlist Management	44
6.4	Trang chủ và tìm kiếm - Homepage & Search	49
6.5	Upload bài hát - Song Upload	52
6.6	Phát nhạc - Player	56
6.7	Nhận diện bài hát - Recognition	58
7	Kiến trúc triển khai (Deployment View)	62
7.1	Tổng quan	62
7.2	Chi tiết các thành phần	62
7.2.1	Client Tier (Tầng Khách hàng)	62
7.2.2	Application Tier (Tầng Ứng dụng)	63
7.2.3	Data Tier (Tầng Dữ liệu)	63
7.3	Các luồng dữ liệu chính (Data Flows)	63
7.3.1	Quy trình Xác thực (Authentication Flow)	63
7.3.2	Các luồng nghiệp vụ khác	63
7.4	Đặc điểm kỹ thuật & Bảo mật	64



8	Kiến trúc phát triển (Development View)	65
8.1	Tổng quan	65
8.2	Tổ chức mã nguồn (Source Code Organization)	66
8.2.1	Phân hệ Frontend (fe/)	66
8.2.2	Phân hệ Backend (be/)	66
8.3	Công nghệ sử dụng (Technology Stack)	67
8.4	Quy trình phát triển (Development Workflow)	67
9	Advanced section	67
9.1	Cloud Deployment and Service (Triển khai và dịch vụ đám mây)	67
9.2	API Integration (Tích hợp API)	68
10	System Implementation	69
10.1	Login Page	69
10.2	Home Page	70
10.3	Account Page	71
10.4	Playlist Page	72
10.5	Upload Page	74
10.6	Liked Page	75
10.7	Search Page	75
10.8	Track Details Page	76
10.9	Artist Details Page	76
10.10	About Page	77
10.11	Lyrics Section	77
10.12	Notification Section	78
10.13	Tracks Queue Section	78
11	Nhận xét, đánh giá và hướng phát triển	78
11.1	Kết quả đạt được	78
11.2	Hạn chế	79
11.3	So sánh tính năng với các nền tảng âm nhạc hiện có	79
11.4	Hướng phát triển trong tương lai	80
11.4.1	Về mặt Công nghệ và Kỹ thuật	80
11.4.2	Về mặt Tính năng và Nội dung	80
11.4.3	Về mặt Thương mại và Quản lý	80



Danh sách thành viên và nhiệm vụ

STT	Họ và tên	MSSV	Nhiệm vụ	% hoàn thành
1	Lư Chấn Vũ	2313955	-	100%
2	Nguyễn Phú Vinh	2313922	-	100%
3	Trần Dương Khiết Nhi	2312509	-	100%
4	Lê Minh Khoa	2311593	-	100%
5	Lê Minh Trí	2313593	-	100%

Bảng 1: Danh sách thành viên và nhiệm vụ

1 Tổng quan về đề tài

1.1 Mô tả đề tài

MusicHUB là nền tảng nghe nhạc trực tuyến, được xây dựng nhằm giải quyết bài toán cân bằng giữa chất lượng âm thanh và trải nghiệm người dùng trong các điều kiện mạng khác nhau. Hệ thống tích hợp công nghệ **Adaptive Bitrate Streaming**, cho phép tự động chuyển đổi linh hoạt giữa các mức chất lượng, đảm bảo quá trình nghe nhạc liền mạch, không gián đoạn ngay cả khi kết nối Internet không ổn định.

Không chỉ dừng lại ở chức năng phát nhạc cơ bản, MusicHUB tập trung vào trải nghiệm cá nhân hóa thông qua thuật toán gợi ý bài hát dựa trên lịch sử nghe và hành vi người dùng. Hệ thống cũng cung cấp các tính năng tương tác nâng cao như **đồng bộ lời bài hát (Lyric Sync)** theo thời gian thực và **nhận diện bài hát qua giai điệu (Audio Fingerprinting)**, giúp người dùng dễ dàng khám phá và tiếp cận nội dung mới.

Đối với các Nghệ sĩ và nhà sáng tạo nội dung, MusicHUB cung cấp một hệ sinh thái quản lý toàn diện, từ việc upload tác phẩm, đến việc theo dõi thống kê chi tiết về lượt nghe và thời gian nghe. Hệ thống được thiết kế với kiến trúc bảo mật, tuân thủ các quy định về bản quyền và có khả năng mở rộng để phục vụ lượng người dùng lớn đồng thời.

1.2 Mục tiêu đề tài

Đề án hướng đến việc xây dựng một hệ thống hoàn chỉnh đáp ứng các tiêu chuẩn kỹ thuật và nghiệp vụ sau:

- **Xây dựng nền tảng phát nhạc ổn định:** Đảm bảo khả năng xử lý file nhạc, hỗ trợ nhiều định dạng và cung cấp tính năng streaming mượt mà với độ trễ thấp ($\leq 800\text{ms}$).
- **Tối ưu hóa trải nghiệm người dùng (UX):** Cung cấp giao diện trực quan, thời gian phản hồi nhanh, độ trễ thấp, trải nghiệm nghe nhạc mượt mà, cung cấp các tính năng nâng cao trải nghiệm nghe của người dùng.
- **Phát triển hệ thống gợi ý và tìm kiếm:** Giúp người dùng dễ dàng tiếp cận nội dung yêu thích thông qua thuật toán gợi ý và công cụ tìm kiếm mạnh mẽ (theo từ khóa, giọng nói/âm thanh).
- **Quản lý nội dung và bản quyền:** Cung cấp công cụ cho Nghệ sĩ tải lên và quản lý bài hát, đồng thời hỗ trợ Quản trị viên kiểm duyệt nội dung để đảm bảo tính hợp pháp và bản quyền.
- **Đảm bảo hiệu năng và bảo mật:** Hệ thống phải chịu tải tốt (hỗ trợ nhiều người dùng đồng thời) và bảo mật thông tin cá nhân người dùng theo các tiêu chuẩn hiện hành.

1.3 Phạm vi đề tài

Hệ thống MusicHUB bao gồm các phân hệ chính và đối tượng sử dụng như sau:

1. Đối tượng sử dụng (Actors):

- **Người dùng (User/Listener):** Nghe nhạc, tạo playlist, tương tác (like, comment), tìm kiếm và nhận diện bài hát.
- **Nghệ sĩ (Artist):** Ngoài các quyền của người dùng, nghệ sĩ có quyền upload bài hát/album, quản lý kho nhạc cá nhân.

- **Quản trị viên (Admin):** Quản lý người dùng, kiểm duyệt nội dung bài hát, xem thống kê hệ thống.

2. Các module chức năng chính:

- **Module Tài khoản:** Đăng ký, đăng nhập (email/social), quản lý thông tin cá nhân, bảo mật tài khoản.
- **Module Nghe nhạc (Player):** Phát nhạc, điều khiển (play/pause/next), hàng chờ (queue), lyric sync, adaptive bitrate.
- **Module Khám phá (Discovery):** Tìm kiếm, bảng xếp hạng (Trending/Top Chart), gợi ý nhạc (Recommendation).
- **Module Thư viện (Library):** Quản lý playlist cá nhân, bài hát yêu thích, lịch sử nghe nhạc.
- **Module Upload & Xử lý media:** Cho phép upload file nhạc, thumbnail, lyric file, xử lý file nhạc sang nhiều bitrate.
- **Module Nhận diện:** Ghi âm và nhận diện bài hát qua so khớp mẫu âm thanh (Fingerprint).

2 Functional Requirements - Non-functional Requirements

2.1 Functional Requirements

2.1.1 Functional Requirements

2.1.1.a Quản lý tài khoản và Xác thực (Authentication & Account Management)

- **FR-AUTH-01:** Hệ thống phải cho phép người dùng đăng ký tài khoản mới bằng email hoặc số điện thoại.
- **FR-AUTH-02:** Hệ thống phải hỗ trợ người dùng đăng nhập thông qua các nhà cung cấp định danh bên thứ ba (Google, Facebook, Apple ID).
- **FR-AUTH-03:** Hệ thống phải cho phép người dùng đã đăng nhập cập nhật thông tin cá nhân (ảnh đại diện, tên hiển thị) và thay đổi mật khẩu.
- **FR-AUTH-04:** Hệ thống phải cung cấp chức năng khôi phục mật khẩu thông qua email xác thực khi người dùng quên mật khẩu.
- **FR-AUTH-05:** Hệ thống phải phân quyền truy cập dựa trên vai trò của tài khoản: Khách (Guest), Người dùng (User) và Quản trị viên (Admin).

2.1.1.b Trình phát nhạc và Điều khiển (Music Player & Controls)

- **FR-PLAY-01:** Hệ thống phải cung cấp giao diện phát nhạc với các chức năng điều khiển cơ bản: Phát (Play), Tạm dừng (Pause), Chuyển bài kế tiếp (Next), và Quay lại bài trước (Previous).
- **FR-PLAY-02:** Hệ thống phải cho phép người dùng thay đổi chế độ phát nhạc, bao gồm: Phát ngẫu nhiên (Shuffle), Lặp lại một bài (Repeat One) và Lặp lại danh sách (Repeat All).

- **FR-PLAY-03:** Hệ thống phải hiển thị lời bài hát (Lyrics) đồng bộ theo thời gian thực (nếu dữ liệu lời bài hát khả dụng) trong quá trình phát nhạc.

2.1.1.c Quản lý danh sách nhạc (Playlist Management)

- **FR-LIST-01:** Hệ thống phải cho phép người dùng tạo các danh sách phát (Playlist) cá nhân mới với tên tùy chọn.
- **FR-LIST-02:** Hệ thống phải cho phép người dùng thêm bài hát đang nghe hoặc bài hát từ kết quả tìm kiếm vào các Playlist hiện có.
- **FR-LIST-03:** Hệ thống phải cho phép người dùng xóa bài hát khỏi Playlist hoặc xóa toàn bộ Playlist khỏi thư viện cá nhân.
- **FR-LIST-04:** Hệ thống phải hiển thị danh sách tất cả các Playlist mà người dùng đã tạo trong mục “Playlists”.

2.1.1.d Khám phá và Tìm kiếm (Discovery & Search)

- **FR-SEARCH-01:** Hệ thống phải cung cấp chức năng tìm kiếm cho phép người dùng nhập từ khóa để truy vấn các bài hát, nghệ sĩ tương ứng.
- **FR-DISC-01:** Hệ thống phải hiển thị danh sách các bài hát thịnh hành (Trending) và các bảng xếp hạng (Top Charts) dựa trên dữ liệu lượt nghe tổng hợp.
- **FR-DISC-02:** Hệ thống phải tích hợp chức năng nhận diện âm nhạc (Audio Fingerprinting), cho phép ghi âm qua microphone thiết bị trong khoảng 15-20 giây để xác định thông tin bài hát đang phát từ môi trường xung quanh.
- **FR-REC-01:** Hệ thống phải tự động tạo danh sách gợi ý bài hát (Recommendation) dựa trên lịch sử nghe nhạc của người dùng.

2.1.1.e Tương tác xã hội (Social Interaction)

- **FR-SOC-01:** Hệ thống phải cho phép người dùng thể hiện sự yêu thích bằng cách nhấn icon “Thích” (Like) bài hát.
- **FR-SOC-02:** Hệ thống phải cho phép người dùng đăng tải và xóa bình luận (Comment) tại trang chi tiết của bài hát.
- **FR-SOC-03:** Hệ thống phải cho phép người dùng theo dõi (Follow) các Nghệ sĩ để nhận thông báo về hoạt động mới.
- **FR-SOC-04:** Hệ thống phải cung cấp chức năng chia sẻ (Share) bài hát hoặc playlist thông qua liên kết hoặc tích hợp với các mạng xã hội.

2.1.1.f Chức năng dành cho Nghệ sĩ và Quản trị (Artist & Admin Features)

- **FR-ART-01:** Hệ thống phải cho phép tài khoản Nghệ sĩ tải lên (Upload) các tệp tin âm thanh (MP3, WAV), ảnh bìa và nhập dữ liệu metadata (tiêu đề, thể loại) cho bài hát mới.
- **FR-ADM-01:** Hệ thống phải cung cấp giao diện quản trị cho phép Admin xem danh sách người dùng và thống kê hệ thống.
- **FR-ADM-02:** Hệ thống phải cho phép Admin kiểm duyệt, ẩn hoặc gỡ bỏ các nội dung vi phạm chính sách hoặc bản quyền.

2.1.2 Non-interactive Requirements

2.1.2.a Tính năng Streaming ở nhiều mức bitrate

Chức năng này được thiết kế nhằm tối ưu trải nghiệm nghe nhạc của Người dùng trên nhiều loại thiết bị và trong các điều kiện mạng khác nhau. Sau khi một bài hát được upload thành công lên hệ thống, máy chủ sẽ tự động tiến hành xử lý và chuyển đổi file gốc sang nhiều phiên bản với các mức bitrate khác nhau (ví dụ: 64kbps, 128kbps, 320kbps).

- **Cách hoạt động:**

- Khi Người dùng phát một bài hát, hệ thống sẽ cung cấp nhiều lựa chọn bitrate khác nhau (64kbps, 128kbps, 320kbps).
 - * 64kbps: Phù hợp cho kết nối mạng yếu, tiết kiệm dữ liệu.
 - * 128kbps: Chất lượng tiêu chuẩn, cân bằng giữa tốc độ tải và chất lượng âm thanh.
 - * 320kbps: Chất lượng cao, dành cho Người dùng muốn trải nghiệm âm nhạc tốt nhất.
- Người dùng có thể chọn thủ công mức bitrate mong muốn, phù hợp với chất lượng mạng và nhu cầu nghe nhạc.
- Hệ thống hỗ trợ **Adaptive Streaming**: trong quá trình nghe, nếu mạng yếu hoặc không ổn định, bitrate sẽ tự động hạ xuống để tránh gián đoạn; khi mạng mạnh trở lại, bitrate được nâng lên để đảm bảo chất lượng âm thanh tốt nhất.
- Các phiên bản nhạc ở nhiều bitrate được tạo sẵn trong quá trình xử lý upload, do đó việc chuyển đổi diễn ra nhanh chóng và mượt mà.

- **Input:**

- File nhạc gốc (định dạng hợp lệ: MP3, WAV, FLAC,...).
- Metadata bài hát (tên, nghệ sĩ, thể loại, ảnh bìa).
- Thông tin cấu hình hệ thống (các mức bitrate cần tạo).

- **Output:**

- Các phiên bản bài hát ở nhiều mức bitrate (64kbps, 128kbps, 320kbps).
- Đường dẫn hoặc ID truy cập các file đã xử lý để phát trực tuyến.

- **Lợi ích:**

- Người dùng có trải nghiệm nghe nhạc mượt mà, ngay cả khi mạng yếu.
- Tối ưu dung lượng lưu trữ và băng thông cho hệ thống.
- Đáp ứng nhu cầu đa dạng: nghe nhạc tiết kiệm dữ liệu hoặc chất lượng cao.

2.1.2.b Tính năng gợi ý bài hát dựa vào lượt nghe gần đây

Chức năng này giúp cá nhân hoá trải nghiệm nghe nhạc của Người dùng. Hệ thống sẽ phân tích lịch sử nghe nhạc gần đây (bài hát, nghệ sĩ, thể loại) để tự động đưa ra danh sách gợi ý phù hợp với sở thích hiện tại của Người dùng. Danh sách gợi ý có thể hiển thị dưới dạng playlist hoặc phần “Đề xuất cho bạn” trên giao diện chính.

- **Cách hoạt động:**

- Hệ thống lưu trữ và theo dõi lịch sử nghe nhạc của Người dùng.
- Dựa trên dữ liệu lượt nghe gần đây, hệ thống áp dụng thuật toán gợi ý (lọc cộng tác, phân tích nội dung hoặc kết hợp).
- Trả về danh sách bài hát, album hoặc Nghệ sĩ có mức độ tương đồng cao.

• **Input:**

- Lịch sử nghe nhạc gần đây (danh sách bài hát đã phát).
- Metadata của bài hát (thể loại, nghệ sĩ, album, nhãn).
- Dữ liệu hành vi Người dùng khác (để tăng độ chính xác).

• **Output:**

- Danh sách gợi ý gồm các bài hát, playlist hoặc Nghệ sĩ liên quan.
- Giao diện hiển thị mục “Gợi ý cho bạn” được cập nhật động.

• **Lợi ích:**

- Giúp Người dùng khám phá thêm nhạc mới phù hợp với sở thích.
- Tăng mức độ gắn bó và thời gian sử dụng ứng dụng.
- Nâng cao trải nghiệm nhờ cá nhân hoá thông minh.

2.1.2.c Tính năng tự động phát bài hát tiếp theo trong playlist/queue

Khi một bài hát trong playlist hoặc queue kết thúc, hệ thống sẽ tự động phát bài hát kế tiếp mà không cần thao tác thủ công từ Người dùng. Điều này giúp trải nghiệm nghe nhạc liền mạch và thuận tiện, đặc biệt khi Người dùng đang nghe nhạc trong lúc làm việc, tập luyện hoặc thư giãn.

• **Cách hoạt động:**

- Khi bài hát hiện tại kết thúc, hệ thống kiểm tra danh sách playlist/queue đang hoạt động.
- Nếu còn bài hát trong danh sách, hệ thống sẽ tự động phát bài kế tiếp theo thứ tự.
- Nếu đến cuối danh sách: Tùy chế độ, có thể dừng phát nhạc, lặp lại playlist, hoặc bật chế độ phát ngẫu nhiên.
- Quá trình chuyển bài diễn ra mượt mà, không tạo khoảng trống âm thanh lớn.

• **Input:**

- Danh sách playlist hoặc queue do Người dùng tạo hoặc hệ thống gợi ý.
- Thiết lập chế độ phát nhạc (bình thường, lặp lại, ngẫu nhiên).

• **Output:**

- Bài hát kế tiếp được phát tự động ngay sau khi bài hiện tại kết thúc.
- Trạng thái phát nhạc được cập nhật trong trình phát và UI của ứng dụng.

• **Lợi ích:**

- Mang lại trải nghiệm nghe nhạc liền mạch, không bị gián đoạn.
- Người dùng không cần thao tác thủ công, thuận tiện trong nhiều ngữ cảnh.
- Hỗ trợ các chế độ phát linh hoạt, phù hợp với nhu cầu nghe nhạc đa dạng.

2.1.2.d Tính năng thông báo khi Nghệ sĩ được follow phát hành bài hát/album mới

Khi Người dùng follow một Nghệ sĩ, hệ thống sẽ theo dõi các hoạt động phát hành của Nghệ sĩ đó. Khi có bài hát hoặc album mới được phát hành, hệ thống sẽ gửi thông báo đến Người dùng để họ có thể thưởng thức ngay lập tức. Điều này giúp tăng mức độ gắn kết giữa Người dùng và Nghệ sĩ, đồng thời nâng cao trải nghiệm khám phá nhạc mới.

- **Cách hoạt động:**

- Hệ thống lưu danh sách Nghệ sĩ mà Người dùng đã follow.
- Khi Nghệ sĩ phát hành bài hát/album mới, hệ thống kiểm tra và xác định những Người dùng đã follow.
- Gửi thông báo qua ứng dụng (push notification).
- Thông báo chứa thông tin cơ bản như tên bài hát/album, ngày phát hành, và liên kết để nghe trực tiếp.

- **Input:**

- Danh sách Nghệ sĩ được Người dùng follow.
- Dữ liệu phát hành bài hát/album mới của Nghệ sĩ.

- **Output:**

- Thông báo gửi đến Người dùng khi có bài hát/album mới.
- Liên kết dẫn trực tiếp đến nội dung nhạc trong ứng dụng.

- **Lợi ích:**

- Người dùng luôn cập nhật kịp thời nhạc mới từ Nghệ sĩ yêu thích.
- Tăng mức độ tương tác và gắn bó giữa Người dùng và nền tảng.
- Giúp Nghệ sĩ tiếp cận nhanh chóng đến fan hâm mộ của mình.

2.1.2.e Tính năng thống kê thời gian nghe và lượt nghe

Hệ thống cung cấp cho Người dùng thống kê chi tiết về hoạt động nghe nhạc của họ, bao gồm tổng thời gian nghe, số lượt nghe theo ngày/tuần/tháng, các bài hát được nghe nhiều nhất, và Nghệ sĩ được yêu thích nhất. Mục tiêu là giúp Người dùng theo dõi thói quen nghe nhạc, đồng thời tăng mức độ gắn bó với ứng dụng thông qua các báo cáo cá nhân hóa.

- **Cách hoạt động:**

- Mỗi lần Người dùng phát một bài hát, hệ thống ghi nhận thời lượng nghe và số lượt nghe.
- Dữ liệu được lưu trữ và cập nhật liên tục trong cơ sở dữ liệu.
- Người dùng có thể xem thống kê dưới dạng biểu đồ, danh sách hoặc báo cáo theo mốc thời gian (ngày/tuần/tháng/năm).
- Hệ thống có thể tạo báo cáo đặc biệt (ví dụ: tổng kết cuối năm) để tăng trải nghiệm Người dùng.

- **Input:**

- Hoạt động nghe nhạc của Người dùng (thời gian phát, bài hát, Nghệ sĩ, playlist).
- Thông tin Người dùng để liên kết dữ liệu thống kê.

- **Output:**

- Báo cáo thống kê: tổng thời gian nghe, số lượt nghe, top bài hát/Nghệ sĩ/album.
- Biểu đồ hoặc bảng dữ liệu trực quan hiển thị trong ứng dụng.

- **Lợi ích:**

- Người dùng có thể theo dõi thói quen nghe nhạc của bản thân.
- Tăng sự gắn bó với ứng dụng thông qua báo cáo cá nhân hóa.
- Tạo cơ sở dữ liệu cho hệ thống gợi ý nhạc chính xác hơn.
- Có thể sử dụng để tổ chức sự kiện đặc biệt (ví dụ: “Top bài hát năm của bạn”).

2.1.2.f Tính năng tạo và đồng bộ lời bài hát (Lyric Sync)

Hệ thống cho phép Người dùng trải nghiệm nghe nhạc với phần hiển thị lời bài hát được đồng bộ theo thời gian phát nhạc (karaoke-style). Khi upload bài hát, Nghệ sĩ hoặc quản trị viên có thể thêm file lyric kèm theo, hoặc hệ thống hỗ trợ nhập thủ công và đồng bộ từng câu hát với thời gian. Mục tiêu là mang đến trải nghiệm nghe nhạc sinh động, giúp Người dùng dễ dàng theo dõi và hát theo bài hát.

- **Cách hoạt động:**

- Khi upload, Người dùng hoặc Nghệ sĩ cung cấp file lời bài hát (.lrc hoặc định dạng hỗ trợ).
- Nếu không có file sẵn, hệ thống cho phép nhập lời và sử dụng công cụ đồng bộ thời gian (timestamp editor).
- Khi phát nhạc, trình phát hiển thị lời bài hát, cuộn và highlight theo đúng nhịp bài hát.

- **Input:**

- File nhạc upload lên hệ thống.
- File lyric (.lrc) hoặc text lyric do Người dùng/Nghệ sĩ nhập vào.
- Timestamp đồng bộ (có thể nhập thủ công hoặc tự động gợi ý).

- **Output:**

- Lời bài hát hiển thị trực quan và đồng bộ theo thời gian phát nhạc.
- Highlight từng dòng lyric theo nhạc.

- **Lợi ích:**

- Nâng cao trải nghiệm nghe nhạc, đặc biệt với Người dùng muốn hát theo.
- Tăng tính chuyên nghiệp, tạo cảm giác gần gũi như ứng dụng karaoke.

2.2 Non-functional Requirements

2.2.1 Hiệu năng (Performance)

Hệ thống phải có khả năng phản hồi nhanh chóng trong mọi tình huống sử dụng phổ biến.

- Thời gian phản hồi cho các thao tác chính (như đăng nhập, tìm kiếm bài hát, phát nhạc, chuyển bài, tạo playlist,...) **không được vượt quá 2 giây** (đối với tối thiểu 95% yêu cầu), ngay cả khi có **1000 người dùng đồng thời** truy cập.
- Thời gian bắt đầu phát nhạc (từ khi ấn nút Play đến khi nghe) ≤ 800 ms trong điều kiện mạng ổn định.
- Tác vụ nhận diện bài hát từ đoạn ghi âm 6-10s phải trả kết quả trong vòng ≤ 2 giây để đảm bảo trải nghiệm mượt mà.

2.2.2 Bảo mật (Security)

Hệ thống cần đảm bảo mức độ bảo mật cao.

- Người dùng phải đăng nhập qua tài khoản riêng (email/social login OAuth 2.0) để đảm bảo xác thực an toàn.
- Dữ liệu cá nhân (tên, email, playlist, lịch sử nghe) và dữ liệu nhạc phải được mã hóa khi lưu trữ (AES-256) và truyền tải (TLS 1.2+).
- Hệ thống áp dụng **role-based access control** (phân quyền): người dùng thường không thể truy cập chức năng quản trị.
- Phiên đăng nhập tự động hết hạn sau **30 phút không hoạt động**.
- Hệ thống có cơ chế giới hạn tốc độ (rate limit) để ngăn chặn brute force và tấn công DDoS:
 - Tối đa 5 lần đăng nhập thất bại trong vòng 15 phút sẽ khóa tài khoản trong 30 phút.
 - Giới hạn tối đa 100 yêu cầu API mỗi phút cho mỗi địa chỉ IP.
 - Mỗi tài khoản người dùng chỉ được phép phát tối đa 3 luồng nhạc đồng thời.
 - Giới hạn tối đa 10 yêu cầu nhận diện bài hát mỗi phút cho mỗi tài khoản người dùng.

2.2.3 Độ tin cậy (Reliability)

Hệ thống phải có khả năng hoạt động liên tục, ổn định và không bị gián đoạn.

- Hệ thống cần đảm bảo **độ sẵn sàng (uptime)** tối thiểu là **99.9%** cho dịch vụ phát nhạc, và tối thiểu **99.5%** cho dịch vụ nhận diện bài hát.
- Cơ chế sao lưu dữ liệu được thực hiện tự động hàng ngày để tránh mất mát thông tin trong trường hợp xảy ra sự cố hệ thống.
- Có thông báo lỗi rõ ràng nếu hệ thống không khả dụng, kèm theo phương án xử lý.
- Khi gặp sự cố, phải có cơ chế **khôi phục dữ liệu trong vòng 24 giờ**.

2.2.4 Khả năng sử dụng (Usability)

- Giao diện người dùng (UI) trực quan, thân thiện, quen thuộc với các biểu tượng phổ biến (Play/Pause, Next, Like, Playlist,...).
- Hỗ trợ **song ngữ (tiếng Việt và tiếng Anh)** để phù hợp với nhiều đối tượng người dùng.
- Người dùng mới có thể làm quen và thành thạo với các chức năng cơ bản (tìm kiếm, phát nhạc, playlist, nhận diện bài hát,...) sau **≤ 15 phút** hướng dẫn.
- Hỗ trợ **đa nền tảng**: trình duyệt web phổ biến (Chrome, Firefox, Edge, Safari – 2 phiên bản gần nhất) và ứng dụng di động (Android 10+, iOS 14+).
- Hỗ trợ **accessibility** (screen reader, contrast mode, điều khiển bàn phím,...) để đảm bảo các đối tượng khuyết tật có thể sử dụng.

2.2.5 Khả năng mở rộng (Scalability)

- Hệ thống có khả năng phục vụ **thêm ít nhất 3000 người dùng/năm** mà không ảnh hưởng hiệu năng.
- Có thể mở rộng, tích hợp thêm các module mới như: gợi ý nhạc bằng AI, podcast,... mà không làm thay đổi cấu trúc hiện tại.
- Các microservice (nghe nhạc, tìm kiếm, nhận diện, gợi ý) có thể scale độc lập trên cloud.

2.2.6 Tính sẵn sàng (Availability)

- Hệ thống cần hoạt động liên tục và vận hành ổn định **24/7** để hỗ trợ người dùng mọi lúc.
- Thời gian bảo trì định kỳ **không quá 4 lần/năm**, mỗi lần ≤ 4 giờ.
- Nếu bảo trì, cần có thông báo trên hệ thống trước ít nhất **24 giờ**.
- Trong thời gian bảo trì, hệ thống phải hiển thị thông báo rõ ràng cho người dùng.

2.2.7 Tính tương thích (Compatibility)

- Ứng dụng tương thích với nhiều thiết bị (điện thoại, máy tính, máy tính bảng) và nhiều chuẩn phát nhạc (HLS/DASH, AAC/Opus).
- Hệ thống hoạt động tốt trên nhiều nền tảng thiết bị khác nhau:
 - Mobile app tương thích với hệ điều hành **Android 10 trở lên** và **iOS 14 trở lên**.
 - Web app tương thích với các trình duyệt hiện đại như **Google Chrome, Mozilla Firefox, Microsoft Edge, Apple Safari** (đối với 2 phiên bản gần nhất).
- Hệ thống tích hợp API với dịch vụ bên ngoài: social login (Google, Facebook), thanh toán (nếu có Premium), API âm nhạc (metadata, lyric).

2.2.8 Tuân thủ pháp lý (Compliance)

Hệ thống cần tuân thủ các quy định về bản quyền âm nhạc, bảo mật dữ liệu và quyền riêng tư.

- Hệ thống phải tuân thủ **Luật An ninh mạng Việt Nam** liên quan đến thu thập, xử lý, lưu trữ và bảo vệ thông tin cá nhân trong lãnh thổ Việt Nam.
- Nếu mở rộng quốc tế, phải đáp ứng chuẩn **GDPR (General Data Protection Regulation)** về bảo mật dữ liệu và quyền riêng tư.
- Đảm bảo bản quyền âm nhạc: chỉ lưu trữ, phát nhạc khi có giấy phép; có cơ chế gỡ bỏ nội dung vi phạm bản quyền theo yêu cầu trong vòng **48 giờ**.

2.2.9 Khả năng kiểm toán (Audit)

- Hệ thống phải ghi lại toàn bộ hoạt động (logs) của người dùng và quản trị viên.
- Dữ liệu log cần được sao lưu định kỳ (ví dụ: ngày 28 hàng tháng).
- Log chỉ được đọc (read-only), không thể chỉnh sửa hay xóa qua giao diện người dùng.

2.2.10 Hiệu quả sử dụng tài nguyên (Efficiency)

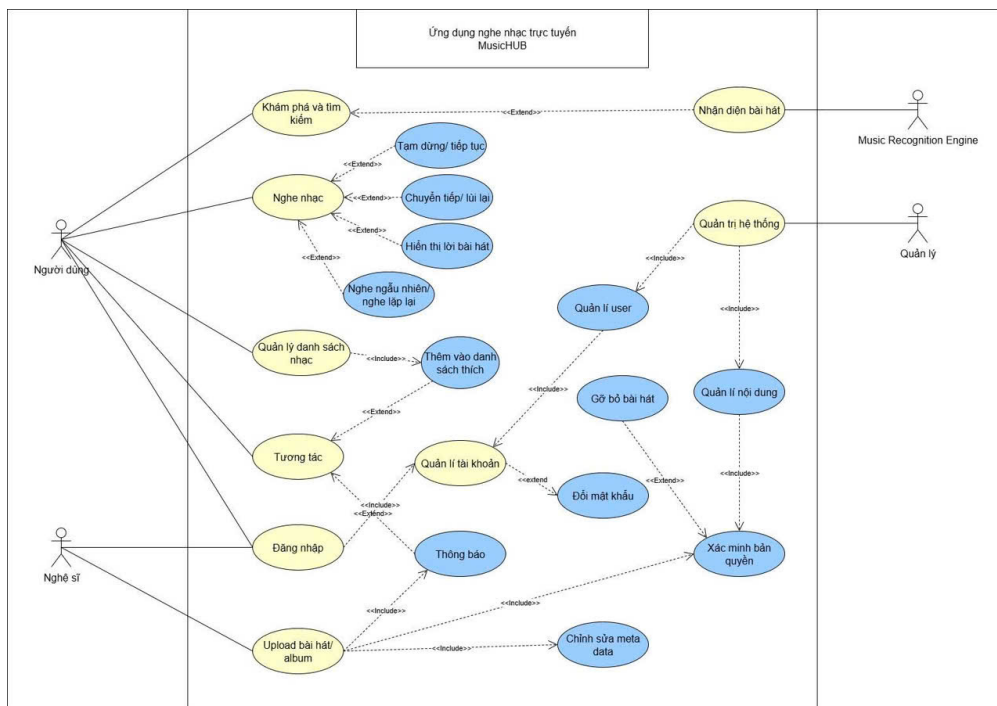
- Hệ thống phải phục vụ ổn định với lưu lượng tối đa khoảng **5000 người dùng hoạt động đồng thời**.
- Tối ưu hóa tài nguyên hệ thống (RAM, CPU) để tránh lãng phí; sử dụng **adaptive bitrate (ABR)** để giảm băng thông khi mạng yếu.
- Ứng dụng mobile tối ưu pin: giảm bitrate khi màn hình tắt (audio-only mode).

2.2.11 Khả năng bảo trì (Maintainability)

- Có **tài liệu kỹ thuật** đầy đủ, rõ ràng (hướng dẫn cài đặt, vận hành, xử lý sự cố).
- Các lỗi phát sinh được ghi log và gửi thông báo cho kỹ thuật viên **trong vòng 5 phút** (qua email/sms).
- Hệ thống cần được kiểm tra và bảo trì định kỳ **tối đa 3 tháng/lần**.
- Kiến trúc phần mềm module hóa, hỗ trợ CI/CD, dễ dàng thêm mới tính năng như gợi ý nhạc AI, podcast,...

3 Use-case Diagram và Use-case Scenario

3.1 Use-case Diagram



Hình 1: Use-case Diagram

3.2 Use-case Scenario

3.2.1 Use-case Upload bài hát/album

Use case name	Upload bài hát/album
Actors	Nghệ sĩ
Description	Chức năng cho phép Người dùng tải lên các bài hát của mình lên hệ thống để lưu trữ, quản lý và chia sẻ với cộng đồng nghe nhạc.
Trigger	Người dùng chọn chức năng “Upload nhạc” trên giao diện hệ thống sau khi đăng nhập thành công.
Pre-Condition(s)	1. Người dùng đã đăng nhập vào hệ thống. 2. Thiết bị của Người dùng được kết nối internet. 3. File nhạc đáp ứng đúng định dạng và dung lượng cho phép (ví dụ: MP3, WAV, dung lượng tối đa 20MB).
Post-Condition(s)	Bài hát được lưu trữ trong hệ thống, hiển thị trong thư viện cá nhân và có thể phát trực tuyến cho Người dùng khác (nếu được chia sẻ công khai).
Normal Flow	1. Người dùng chọn vào phần “Upload nhạc”. 2. Hệ thống hiển thị form tải nhạc (chọn file nhạc, nhập tiêu đề, Nghệ sĩ, thể loại, ảnh bìa,...). 3. Người dùng nhập thông tin và chọn file nhạc từ thiết bị. 4. Người dùng nhấn nút “Upload”. 5. Hệ thống tiến hành tải file nhạc lên server. 6. Sau khi upload thành công, hệ thống thông báo và hiển thị bài hát trong thư viện cá nhân.
Exception Flow	3a. Nếu file nhạc sai định dạng hoặc vượt quá dung lượng cho phép, hệ thống thông báo lỗi. 5a. Nếu kết nối internet bị gián đoạn trong khi tải lên, hệ thống hiển thị thông báo thất bại và yêu cầu thử lại. 6a. Nếu hệ thống lỗi trong quá trình xử lý file, hiển thị thông báo “Upload thất bại”.
Alternative Flow	3b. Người dùng có thể hủy thao tác upload bất kỳ lúc nào để quay lại thư viện. 4b. Người dùng có thể chọn chế độ chia sẻ: công khai, riêng tư. 5b. Hệ thống hỗ trợ upload nhiều bài hát cùng lúc để tiết kiệm thời gian.

Bảng 2: Mô tả use-case Upload bài hát/album

3.2.2 Use-case Đăng nhập & Đăng ký

Use case name	Đăng nhập & Đăng ký
Actors	Người dùng, Nghệ sĩ
Description	Người dùng có thể tạo tài khoản mới (Đăng ký) hoặc truy cập tài khoản đã có (Đăng nhập) để sử dụng ứng dụng.
Trigger	Người dùng mở web và chọn tính năng “Đăng nhập / Đăng ký”.
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Người dùng chưa có tài khoản (nếu đăng ký) hoặc đã có tài khoản hợp lệ (nếu đăng nhập).
Post-Condition(s)	Người dùng đăng nhập thành công hoặc tạo mới tài khoản thành công và được chuyển vào giao diện chính, tài khoản đăng ký thành công sẽ được thêm vào hệ thống.
Normal Flow	Đăng nhập: 1. Người dùng chọn “Đăng nhập”. 2. Nhập email/số điện thoại/tên đăng nhập và mật khẩu. 3. Hệ thống kiểm tra thông tin xác thực. 4. Nếu đúng → Đăng nhập thành công và hiển thị trang chủ. Đăng ký: 1. Người dùng chọn “Đăng ký”. 2. Nhập thông tin cá nhân: email/số điện thoại, mật khẩu, tên hiển thị. 3. Xác thực tài khoản qua email. 4. Hệ thống tạo tài khoản mới và hiển thị thông báo thành công.
Exception Flow	Đăng nhập: 2a. Bỏ trống trường thông tin → báo lỗi “Vui lòng nhập đầy đủ thông tin”. 3a. Sai mật khẩu → báo lỗi “Sai mật khẩu”. 3b. Tài khoản bị khóa → báo lỗi “Tài khoản của bạn đã bị khóa”. Đăng ký: 2a. Email/số điện thoại đã tồn tại → báo lỗi “Tài khoản đã tồn tại”. 2b. Mật khẩu yếu/không hợp lệ → báo lỗi “Mật khẩu không hợp lệ”. 3a. Không xác thực được email → báo lỗi “Xác thực thất bại”.
Alternative Flow	Đăng nhập: 2b. Người Dùng chọn “Đăng nhập bằng Google/Facebook/Apple ID”. 2c. Người Dùng chọn “Quên mật khẩu” → chuyển sang luồng khôi phục mật khẩu. Đăng ký: 2c. Người Dùng có thể tải ảnh đại diện hoặc chọn sở thích âm nhạc ngay khi tạo tài khoản.

Bảng 3: Mô tả use-case Đăng nhập & Đăng ký

3.2.3 Use-case Quản lý tài khoản

Use case name	Quản lý tài khoản
Actors	Người dùng, Nghệ sĩ
Description	Người dùng có thể quản lý tài khoản cá nhân bao gồm: đăng ký, đổi mật khẩu, cập nhật thông tin cá nhân (avatar, tên hiển thị, email), và xóa tài khoản.
Trigger	Người dùng chọn chức năng “Quản lý tài khoản” trên giao diện hệ thống sau khi đăng nhập thành công.
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Người dùng đã đăng nhập thành công.
Post-Condition(s)	Hệ thống cập nhật thông tin tài khoản theo thao tác của Người Dùng.
Normal Flow	1. Người dùng chọn vào phần “Quản lý tài khoản” trong giao diện hệ thống. 2. Hệ thống hiển thị thông tin tài khoản hiện tại. 3. Người dùng chọn hành động: chỉnh sửa thông tin cá nhân / đổi mật khẩu / thiết lập bảo mật. 4. Người dùng nhập thông tin mới hoặc thay đổi cần thiết. 5. Người dùng xác nhận và lưu thay đổi. 6. Hệ thống cập nhật dữ liệu và thông báo thành công.
Exception Flow	3a. Nếu hệ thống không tải được thông tin tài khoản thì hiển thị lỗi “Không thể tải dữ liệu”. 4a. Nếu thông tin nhập sai định dạng (ví dụ email không hợp lệ, mật khẩu quá ngắn), hệ thống thông báo lỗi và yêu cầu nhập lại. 5a. Nếu kết nối internet bị gián đoạn trong khi lưu, hệ thống hiển thị thông báo thất bại và yêu cầu thử lại.
Alternative Flow	3b. Người dùng có thể hủy thao tác và quay lại trang chính mà không thay đổi gì. 4b. Người dùng có thể bật/tắt các tính năng nâng cao như xác thực 2 lớp.

Bảng 4: Mô tả use-case Quản lý tài khoản

3.2.4 Use-case Tương tác

Use case name	Tương tác
Actors	Người dùng
Description	Người dùng có thể thực hiện các hành động tương tác trong hệ thống như: thích (like) bài hát, bình luận bài hát, theo dõi nghệ sĩ, chia sẻ bài hát, nghệ sĩ, playlist lên mạng xã hội.
Trigger	Người dùng chọn một bài hát, nghệ sĩ hoặc playlist bất kỳ trong hệ thống và mở giao diện chi tiết.
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Người dùng đã đăng nhập thành công. 3. Bài hát, nghệ sĩ đã có sẵn trên hệ thống.
Post-Condition(s)	Hệ thống lưu lại thông tin tương tác (like, bình luận, theo dõi, chia sẻ) và cập nhật dữ liệu liên quan.
Normal Flow	1. Người dùng chọn một bài hát, nghệ sĩ, playlist. 2. Hệ thống hiển thị chi tiết bài hát, nghệ sĩ, playlist. 3. Người dùng chọn hành động muốn thực hiện: 3a. Nhấn nút “Thích” (Like) để thêm vào danh sách yêu thích. 3b. Viết và gửi bình luận cho bài hát. 3c. Nhấn “Theo dõi” để theo dõi nghệ sĩ. 3d. Chọn “Chia sẻ” và lựa chọn kênh chia sẻ (Các trang mạng xã hội). 4. Hệ thống xác nhận và cập nhật thông tin tương tác.
Exception Flow	2a. Nếu bài hát, nghệ sĩ, playlist không tồn tại hoặc bị xóa → hiển thị thông báo lỗi. 3b1. Nếu bình luận chứa nội dung vi phạm chính sách → hệ thống từ chối đăng và hiển thị thông báo. 3d1. Nếu chia sẻ thất bại do mất kết nối internet → hiển thị thông báo “Chia sẻ không thành công, vui lòng thử lại”.
Alternative Flow	3a1. Người dùng có thể “Bỏ thích” (Unlike) nếu trước đó đã thích. 3b2. Người dùng có thể chỉnh sửa hoặc xóa bình luận đã đăng. 3c1. Người dùng có thể hủy theo dõi nghệ sĩ. 3d2. Người dùng có thể sao chép đường dẫn (link) thay vì chia sẻ trực tiếp.

Bảng 5: Mô tả use-case Tương tác

3.2.5 Use-case Nghe nhạc

Use case name	Nghe nhạc
Actors	Người dùng
Description	Người dùng có thể phát và điều khiển nhạc với nhiều tính năng nâng cao: phát/tạm dừng, tiếp tục, chuyển tiếp/lùi lại, bật chế độ nghe ngẫu nhiên, nghe lặp lại (một bài hoặc toàn bộ danh sách), thêm bài vào hàng chờ, cài giờ tắt nhạc (sleep timer), và xem lời bài hát.
Trigger	Người dùng chọn một bài hát, playlist, album và nhấn nút “Phát nhạc”.
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Ứng dụng được cấp quyền truy cập âm thanh. 3. Người dùng đã đăng nhập thành công. 4. Bài hát đã có sẵn trên hệ thống và có thể phát.
Post-Condition(s)	Hệ thống phát nhạc theo thao tác của Người dùng, cập nhật trạng thái trình phát (player state) và lưu lại lịch sử nghe nhạc.
Normal Flow	1. Người dùng chọn một bài hát, playlist, album. 2. Hệ thống tải dữ liệu nhạc và bắt đầu phát. 3. Người dùng có thể thực hiện các thao tác điều khiển: 3a. Nhấn “Tạm dừng” (Pause) hoặc “Tiếp tục” (Play). 3b. Nhấn “Chuyển tiếp” (Next) hoặc “Lùi lại” (Previous). 3c. Bật chế độ “Nghe ngẫu nhiên” (Shuffle). 3d. Chọn chế độ “Nghe lặp lại” (Repeat one / Repeat all). 3e. Thêm bài hát vào hàng chờ phát (Queue). 3f. Cài đặt “Giờ tắt nhạc” (Sleep timer). 3g. Xem lời bài hát (Lyrics) nếu có sẵn. 4. Hệ thống phản hồi ngay lập tức và cập nhật trình phát nhạc. 5. Khi bài hát kết thúc, hệ thống tự động phát bài tiếp theo trong playlist/queue hoặc dừng lại nếu không còn bài nào
Exception Flow	2a. Nếu bài hát không thể phát (do bản quyền hoặc lỗi file) → hiển thị thông báo. 2b. Nếu người dùng bị mất kết nối trong lúc đang phát nhạc → hiển thị thông báo “Mất kết nối, vui lòng kiểm tra lại” và tạm dừng phát nhạc. 3g1. Nếu bài hát không có lời (lyrics) trong cơ sở dữ liệu → hiển thị “Chưa có lời bài hát”.
Alternative Flow	3a1. Người dùng có thể sử dụng tai nghe hoặc thiết bị ngoài để điều khiển (nút Play/Pause). 3e1. Người dùng có thể sắp xếp lại thứ tự bài hát trong hàng chờ. 3f2. Người dùng có thể hủy hoặc thay đổi thời gian sleep timer.

Bảng 6: Mô tả use-case Nghe nhạc

3.2.6 Use-case Khám phá và tìm kiếm

Use case name	Khám phá và tìm kiếm
Actors	Người dùng
Description	Người dùng có thể tìm kiếm bài hát, nghệ sĩ, album, playlist và khám phá nhạc mới thông qua bảng xếp hạng, xu hướng (trending) và gợi ý cá nhân hóa.
Trigger	Người dùng mở thanh tìm kiếm hoặc tab “Khám phá” trong ứng dụng.
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Người dùng đã đăng nhập thành công. 3. Dữ liệu về bài hát, nghệ sĩ, album phải có trong cơ sở dữ liệu.
Post-Condition(s)	Hệ thống trả về kết quả tìm kiếm hoặc danh sách nhạc khám phá phù hợp, cho phép Người dùng chọn và nghe nhạc ngay.
Normal Flow	1. Người dùng mở tính năng Tìm kiếm/Khám phá. 2. Người dùng có thể thực hiện: 2a. Nhập từ khóa để tìm bài hát, nghệ sĩ, album, playlist. 2b. Xem danh sách Trending hoặc Top Chart. 2c. Nhận gợi ý nhạc cá nhân hóa dựa trên lịch sử nghe, lượt thích và nghệ sĩ theo dõi. 3. Hệ thống hiển thị danh sách kết quả. 4. Người dùng chọn nội dung mong muốn (phát nhạc, xem chi tiết nghệ sĩ, album, playlist).
Exception Flow	2a1. Nếu không tìm thấy kết quả → hiển thị thông báo “Không tìm thấy nội dung phù hợp”. 2a2. Nếu không nhập gì → hiển thị “Vui lòng nhập từ khóa”. 2a3. Nếu kết nối internet bị gián đoạn trong khi tìm kiếm → hiển thị thông báo “Mất kết nối, vui lòng kiểm tra lại”.
Alternative Flow	2a2. Người dùng có thể dùng bộ lọc nâng cao (theo thể loại, năm phát hành, nghệ sĩ, album...). 2b2. Người dùng có thể chọn xem bảng xếp hạng theo từng khu vực/quốc gia. 2c2. Người dùng có thể cập nhật gợi ý bằng cách thay đổi sở thích cá nhân (genres, mood...).

Bảng 7: Mô tả use-case Khám phá và tìm kiếm

3.2.7 Use-case Nhận diện bài hát

Use case name	Nhận diện bài hát
Actors	Người dùng
Description	Người dùng bật tính năng nhận diện nhạc để hệ thống nghe một đoạn nhạc từ môi trường xung quanh, sau đó so sánh với cơ sở dữ liệu fingerprint để xác định bài hát.
Trigger	Người dùng nhấn vào biểu tượng "Nhận diện bài hát" trên thanh tìm kiếm.
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Ứng dụng được cấp quyền truy cập micro. 3. Hệ thống có cơ sở dữ liệu nhạc đủ lớn để so khớp.
Post-Condition(s)	Hệ thống hiển thị thông tin bài hát (tên, nghệ sĩ, album) và cung cấp tùy chọn phát nhạc hoặc lưu vào playlist.
Normal Flow	1. Người dùng chọn tính năng Nhận diện bài hát. 2. Hệ thống yêu cầu quyền truy cập micro và người dùng cấp phép. 3. Hệ thống ghi lại một đoạn âm thanh từ 15-20s. 4. Hệ thống trích xuất "Fingerprint" (dấu vân tay âm thanh) từ đoạn ghi âm để so khớp với dữ liệu. 5. Hệ thống trả về kết quả gồm tên bài hát, nghệ sĩ, album (nếu có). 6. Người dùng có thể chọn nghe bài hát, thêm vào playlist hoặc chia sẻ.
Exception Flow	2a. Người dùng từ chối quyền micro → hệ thống báo không thể sử dụng tính năng. 3a. Không thu được âm thanh do quá ồn, micro hỏng → hệ thống yêu cầu thử lại. 5a. Nếu không tìm thấy kết quả khớp → hệ thống thông báo “Không nhận diện được bài hát”. 5b. Nếu có lỗi kết nối trong quá trình so khớp → hệ thống báo “Mất kết nối” và cho phép thử lại sau.
Alternative Flow	5c. Người dùng có thể tìm kiếm bài hát thủ công.

Bảng 8: Mô tả use-case Nhận diện bài hát

3.2.8 Use-case Quản lý danh sách nhạc

Use case name	Quản lý danh sách nhạc (playlist)
Actors	Người dùng
Description	Cho phép người dùng xem, thêm, sửa và xóa bài hát trong danh sách nhạc của họ.
Trigger	Người dùng truy cập danh sách nhạc và chọn một chức năng quản lý danh sách nhạc (tạo danh sách, thêm bài hát, xóa bài hát, xóa danh sách, .v.v).
Pre-Condition(s)	1. Thiết bị của Người dùng phải được kết nối internet. 2. Người dùng đã đăng nhập thành công. 3. Hệ thống có ít nhất một bài hát.
Post-Condition(s)	1. Danh sách nhạc của User được cập nhật theo hành động đã thực hiện (thêm, sửa, xóa). 2. Người dùng nhận được thông báo về kết quả của hành động.
Normal Flow	1. Người dùng truy cập vào phần “Playlists” trong giao diện hệ thống. 2. Hệ thống hiển thị danh sách nhạc của người dùng hiện tại. 3. Người dùng thực hiện thao tác điều khiển: 3a. Người dùng chọn “Create new playlist”, hệ thống hiển thị yêu cầu điền tên playlist, người dùng nhập và nhấn xác nhận. 3b. Người dùng chọn “Add to playlist” với bài hát muốn thêm, hệ thống yêu cầu chọn playlist muốn thêm, người dùng chọn và nhấn xác nhận. 3c. Người dùng chọn nút xóa tại bài hát muốn xóa trong playlist, hệ thống hiển thị thông báo xác nhận, người dùng xác nhận. 3d. Người dùng chọn “Delete” tại playlist muốn xóa, hệ thống hiển thị thông báo xác nhận, người dùng xác nhận. 4. Hệ thống ghi nhận và cập nhật danh sách playlist của người dùng.
Exception Flow	1. Nếu lỗi xảy ra (mất kết nối máy chủ, hệ thống bận, .v.v), mọi yêu cầu từ người dùng đều bị từ chối và hệ thống sẽ hiển thị thông báo lỗi. 3a. Nếu người dùng nhập tên playlist đã tồn tại, hệ thống thông báo lỗi và yêu cầu người dùng nhập lại. 3b. Nếu người dùng thêm 1 bài hát đã tồn tại trong playlist, hệ thống hiển thị thông báo đã tồn tại và không cập nhật bài hát.
Alternative Flow	3a. Người dùng không đặt tên mà trực tiếp xác nhận, hệ thống vẫn sẽ ghi nhận và đặt tên theo format mặc định “Unnamed Playlist”. 3c/3d. Nếu người dùng không xác nhận, hệ thống hoàn tác toàn bộ thao tác và cập nhật lại trạng thái ban đầu của danh sách playlist.

Bảng 9: Mô tả use-case Quản lý danh sách nhạc

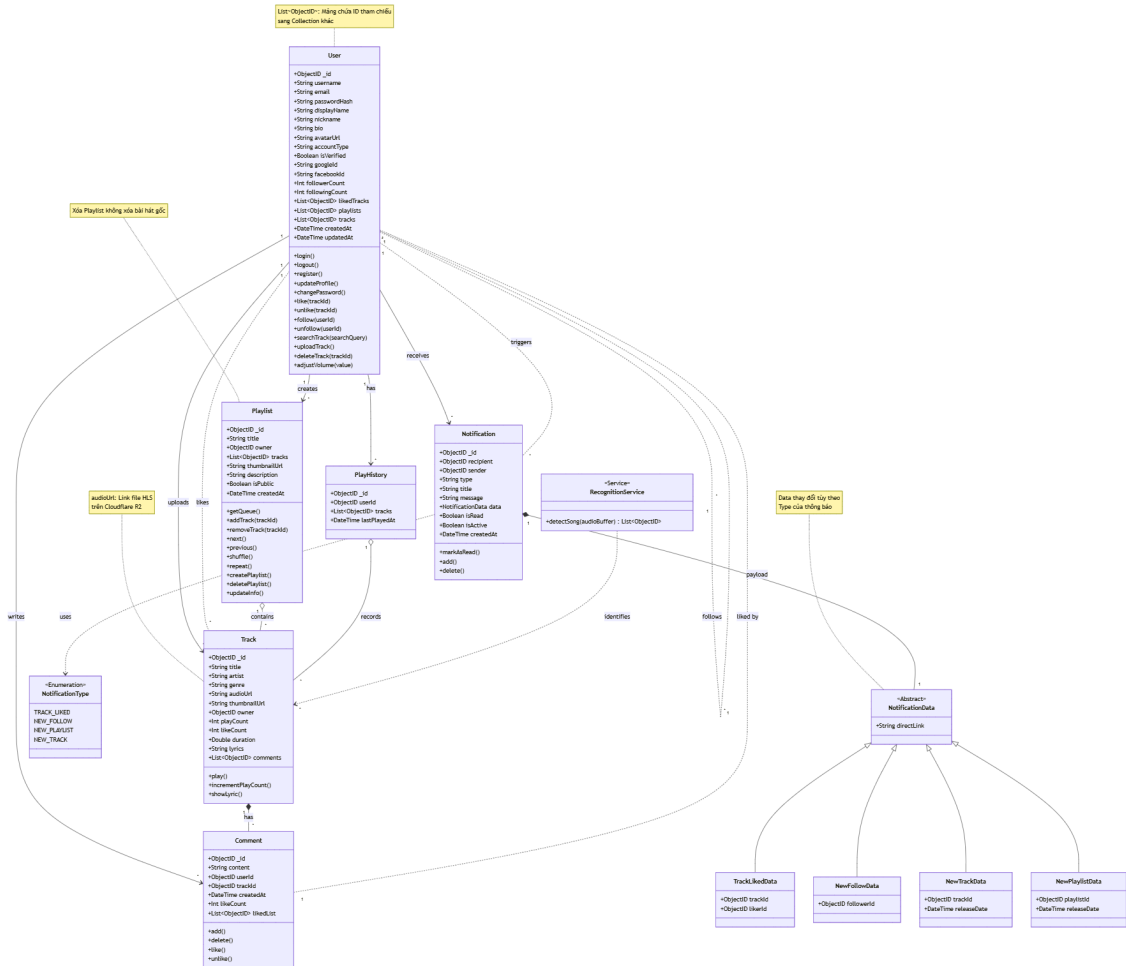
3.2.9 Use-case Quản trị hệ thống

Use case name	Quản trị hệ thống
Actors	Quản lý
Description	Cho phép quản lý thực hiện các tác vụ quản trị để duy trì và điều hành hệ thống.
Trigger	Quản lý truy cập vào trang quản lý hệ thống trong giao diện quản lý.
Pre-Condition(s)	1. Thiết bị của quản lý phải được kết nối internet. 2. Quản lý đã đăng nhập thành công dưới quyền quản trị .
Post-Condition(s)	1. Các thay đổi về người dùng, nội dung, hoặc bản quyền đã được cập nhật thành công trong hệ thống. 2. Hệ thống duy trì trạng thái ổn định và các quy định được tuân thủ.
Normal Flow	1. Quản lý truy cập vào trang quản lý hệ thống với tài khoản có quyền quản trị. 2. Hệ thống điều hướng đến trang quản trị hệ thống. 3. Quản lý có thể lựa chọn hành động: 3a. Xem, sửa (đối với quyền truy cập), xóa người dùng. 3b. Phê duyệt, gỡ bỏ nội dung, bài hát, album công khai thủ công. 3c. Thay đổi các thông tin của nội dung phù hợp với chính sách. 3d. Kiểm tra tính minh bạch và bản quyền của nội dung đăng tải. 4. Hệ thống thông báo thao tác thành công. 5. Hệ thống tự động ghi log đảm bảo tính minh bạch về sau.
Exception Flow	3.1. Nếu quản lý cố gắng truy cập một chức năng không có quyền, hệ thống sẽ thông báo lỗi "Bạn không có quyền truy cập chức năng này". 3.2. Nếu có lỗi trong quá trình cập nhật dữ liệu (thông tin không hợp lệ, .v.v), hệ thống sẽ hiển thị thông báo lỗi và không thực hiện thay đổi. 3.3. Nếu lỗi xảy ra (mất kết nối máy chủ, hệ thống bận, .v.v), mọi thông tin thay đổi sẽ được lưu tạm thời và hệ thống sẽ hiển thị thông báo "Đang trong quá trình bảo trì" cho tới khi có kết nối lại và tự động thực hiện lại thao tác.
Alternative Flow	1a. Tài khoản không có quyền thì hệ thống tự động điều hướng đến giao diện người dùng bình thường. 3a/3b/3c/3d. Nếu thao tác không được xác nhận, hệ thống hoàn tác toàn bộ thao tác và cập nhật lại trạng thái ban đầu của nội dung.

Bảng 10: Mô tả use-case Quản trị hệ thống

4 Phân tích hệ thống

4.1 Class Diagram



Hình 2: Class Diagram

4.2 Mô tả phương thức

4.2.1 Nhóm lớp Người dùng (User Group)

Nhóm này quản lý các tác vụ liên quan đến tài khoản, xác thực và các hành vi tương tác trực tiếp của người dùng với hệ thống.

Tên lớp	Phương thức	Mô tả chức năng
User	login()	Xác thực người dùng (Local/OAuth).
	logout()	Đăng xuất và xóa phiên làm việc.
	register()	Đăng ký tài khoản mới vào hệ thống.
	updateProfile()	Cập nhật thông tin cá nhân (Avatar, Nick-name, Bio).
	changePassword()	Đổi mật khẩu đăng nhập.
	like(trackId)	Thích bài hát.
	unlike(trackId)	Bỏ thích bài hát.
	follow(userId)	Theo dõi nghệ sĩ hoặc người dùng khác.
	unfollow(userId)	Hủy theo dõi.
	searchTrack(searchQuery)	Tìm kiếm bài hát theo tên hoặc nghệ sĩ (input từ thanh header).
	uploadTrack()	Xử lý tải lên file mp3, convert sang HLS và lưu metadata.
	deleteTrack(trackId)	Xóa bài hát khỏi hệ thống (yêu cầu quyền sở hữu).
	adjustVolume(value)	Điều chỉnh âm lượng của trình phát nhạc.

Bảng 11: Mô tả các phương thức nhóm lớp Người dùng

4.2.2 Nhóm lớp Nghiệp vụ và Dữ liệu (Business & Data)

Nhóm này chứa các thực thể cốt lõi của ứng dụng nghe nhạc, bao gồm quản lý bài hát, danh sách phát và tương tác cộng đồng.

Tên lớp	Phương thức	Mô tả chức năng
Track	play()	Phát bài hát (streaming HLS) và kích hoạt ghi nhận lịch sử.
	incrementPlayCount()	Tăng số lượt nghe trong cơ sở dữ liệu.
	showLyric()	Hiển thị lời bài hát (nếu có file .lrc).
Playlist	createPlaylist()	Tạo danh sách phát mới.
	deletePlaylist()	Xóa danh sách phát hiện tại.
	addTrack(trackId)	Thêm bài hát vào playlist.
	removeTrack(trackId)	Xóa bài hát khỏi playlist.
	updateInfo()	Cập nhật tên, mô tả, ảnh bìa playlist.
	getQueue()	Lấy danh sách hàng đợi các bài hát sắp phát.
	next()	Chuyển sang bài hát kế tiếp trong hàng đợi.
	previous()	Quay lại bài hát trước đó.
	shuffle()	Xáo trộn ngẫu nhiên danh sách phát.
	repeat()	Lặp lại danh sách hoặc bài hát hiện tại.



Comment	<code>add()</code>	Đăng tải bình luận mới cho bài hát.
	<code>delete()</code>	Xóa bình luận.
	<code>like()</code>	Thích một bình luận.
	<code>unlike()</code>	Bỏ thích bình luận.

Bảng 12: Mô tả các phương thức nhóm lớp Nghiệp vụ và Dữ liệu

4.2.3 Nhóm lớp Hệ thống và Dịch vụ (System & Services)

Nhóm này xử lý các tác vụ nền tảng như thông báo thời gian thực và tích hợp API nhận diện âm thanh.

Tên lớp	Phương thức	Mô tả chức năng
Notification	<code>add()</code>	Tạo thông báo mới (khi có sự kiện like, follow, upload).
	<code>delete()</code>	Xóa thông báo khỏi danh sách.
	<code>markAsRead()</code>	Đánh dấu thông báo đã được xem.
Recognition Service	<code>detectSong(audioBuffer)</code>	Gửi mẫu âm thanh đến API (Shazam) để nhận diện và trả về danh sách bài hát khớp trong hệ thống.

Bảng 13: Mô tả các phương thức nhóm lớp Hệ thống và Dịch vụ

5 Thiết kế Cơ sở dữ liệu

Dữ liệu được tổ chức dưới dạng Document (MongoDB). Dưới đây là chi tiết các Schema:

5.1 User Schema

Schema này lưu trữ thông tin tài khoản và hồ sơ người dùng.


```
1  const userSchema = new mongoose.Schema(  
2    {  
3      username: { type: String, required: true, unique: true },  
4      email: { type: String, required: true, unique: true },  
5      password: {  
6        type: String,  
7        required: function() {  
8          return !this.googleId && !this.facebookId;  
9        }  
10     },  
11     nickname: { type: String },  
12     thumbnailUrl: { type: String },  
13     bio: { type: String },  
14     country: { type: String },  
15     followerCount: { type: Number, default: 0 },  
16     followingCount: { type: Number, default: 0 },  
17     likedTracks: [{ type: mongoose.Schema.Types.ObjectId, ref: "Track" }],  
18     playlists: [{ type: mongoose.Schema.Types.ObjectId, ref: "Playlist" }],  
19     tracks: [{ type: mongoose.Schema.Types.ObjectId, ref: "Track" }],  
20     googleId: { type: String },  
21     facebookId: { type: String },  
22     isVerified: { type: Boolean, default: false },  
23     accountType: {  
24       type: String,  
25       enum: ['manual', 'google', 'facebook', 'hybrid'],  
26       default: 'manual'  
27     },  
28   },  
29   { timestamps: true }  
30 );
```

Hình 3: Sơ đồ User Schema

STT	Trường	Mô tả
1	username	Tên đăng nhập duy nhất của người dùng
2	email	Địa chỉ email duy nhất
3	password	Mật khẩu (bắt buộc nếu không dùng Google/Facebook)
4	nickname	Tên hiển thị của người dùng
5	accountType	Loại tài khoản (manual, google, facebook, hybrid)
6	isVerified	Trạng thái xác thực tài khoản (Boolean)
7	followerCount	Số lượng người theo dõi
8	followingCount	Số lượng người đang theo dõi

5.2 Track Schema

Schema lưu trữ thông tin về bài hát được tải lên hệ thống.

```
1  const TrackSchema = new mongoose.Schema(  
2    {  
3      title: { type: String, required: true },  
4      artist: { type: String, required: true },  
5      genre: { type: String },  
6      duration: { type: Number, required: true, default: 0 }, // in seconds  
7      audioUrl: { type: String, required: true, unique: true },  
8      thumbnailUrl: { type: String, required: true, unique: true },  
9      owner: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
10     playCount: { type: Number, default: 0 },  
11     likeCount: { type: Number, default: 0 },  
12     commentCount: { type: Number, default: 0 },  
13     comments: { type: mongoose.Schema.Types.ObjectId, ref: "Comments" },  
14     status: { type: String, enum: ["public", "private"], default: "public" },  
15     tags: [{ type: String }],  
16     lyrics: {  
17       type: String,  
18       default: ""  
19     },  
20   },  
21   { timestamps: true }  
22 );
```

Hình 4: Sơ đồ Track Schema

STT	Trường	Mô tả
1	title	Tên bài hát
2	artist	Tên nghệ sĩ thể hiện
3	audioUrl	Đường dẫn đến file âm thanh gốc
4	thumbnailUrl	Đường dẫn ảnh bìa bài hát
5	owner	ID người dùng (User) tải bài hát lên
6	status	Trạng thái hiển thị (public/private)
7	lyrics	Lời bài hát
8	playCount	Tổng số lượt phát
9	likeCount	Tổng số lượt thích
10	comments	Danh sách comments

5.3 Playlist Schema

Schema lưu trữ thông tin danh sách phát nhạc.

```
1  const PlaylistsSchema = new mongoose.Schema(  
2    {  
3      title: { type: String, required: true },  
4      description: { type: String },  
5      thumbnailUrl: { type: String },  
6      owner: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
7      tracks: [{ type: mongoose.Schema.Types.ObjectId, ref: "Track" }],  
8      likeCount: { type: Number, default: 0 },  
9      playCount: { type: Number, default: 0 },  
10     commentCount: { type: Number, default: 0 },  
11     status: { type: String, enum: ["public", "private"], default: "public" },  
12     comments: { type: mongoose.Schema.Types.ObjectId, ref: "Comments" },  
13   },  
14   { timestamps: true }  
15 );
```

Hình 5: Sơ đồ Playlist Schema

STT	Trường	Mô tả
1	title	Tên danh sách phát
2	description	Mô tả ngắn về danh sách phát
3	owner	ID người sở hữu danh sách phát
4	tracks	Mảng chứa ID các bài hát (Track) trong playlist
5	status	Trạng thái công khai hoặc riêng tư
6	likeCount	Số lượt thích playlist
7	thumbnailUrl	Đường dẫn ảnh bìa danh sách phát

5.4 Notification Schema

Schema quản lý các thông báo gửi đến người dùng.

```
1  const NotificationSchema = new mongoose.Schema(  
2    {  
3      recipient: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
4      sender: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
5      type: {  
6        type: String,  
7        enum: ["new_track", "new_playlist", "new_follow", "track_liked"],  
8        required: true  
9      },  
10     title: { type: String, required: true },  
11     message: { type: String, required: true },  
12     data: {  
13       trackId: { type: mongoose.Schema.Types.ObjectId, ref: "Track" },  
14       playlistId: { type: mongoose.Schema.Types.ObjectId, ref: "Playlist" },  
15       followerId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },  
16       likerId: { type: mongoose.Schema.Types.ObjectId, ref: "User" },  
17       releaseDate: { type: Date },  
18       directLink: { type: String }  
19     },  
20     isRead: { type: Boolean, default: false },  
21     isActive: { type: Boolean, default: true }  
22   },  
23   { timestamps: true }  
24 );
```

Hình 6: Sơ đồ Notification Schema

STT	Trường	Mô tả
1	recipient	ID người nhận thông báo
2	sender	ID người gửi thông báo
3	type	Loại thông báo (new_track, new_follow, v.v.)
4	message	Nội dung hiển thị của thông báo
5	isRead	Trạng thái đã đọc (Boolean)
6	data	Object chứa ID liên quan (trackId, playlistId...)

5.5 Comment Schema

Schema quản lý các bình luận của người dùng.

```
1  const CommentsSchema = new mongoose.Schema(  
2    {  
3      comments: [  
4        {  
5          message: { type: mongoose.Schema.Types.ObjectId, ref: "Comment" },  
6        },  
7      ],  
8    },  
9    { timestamps: true }  
10 );  
11  
12 const CommentSchema = new mongoose.Schema(  
13   {  
14     owner: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
15     content: { type: String, required: true },  
16     likeCount: { type: Number, default: 0 },  
17     likedList: [  
18       {  
19         type: mongoose.Schema.Types.ObjectId,  
20         ref: "User",  
21       },  
22     ],  
23     timeline: { type: Number },  
24     replies: { type: mongoose.Schema.Types.ObjectId, ref: "Comments" },  
25   },  
26   { timestamps: true }  
27 );
```

Hình 7: Sơ đồ Comment Schema

STT	Trường	Mô tả
1	owner	ID người viết bình luận
2	content	Nội dung bình luận
3	likeCount	Số lượt thích của bình luận
4	replies	ID tham chiếu đến các câu trả lời (nested comments)
5	timeline	Thời điểm bình luận trong bài hát (giây)

5.6 Follow Schema

Schema quản lý mối quan hệ theo dõi giữa người dùng.

```
1 const FollowSchema = new mongoose.Schema(  
2   {  
3     follower: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
4     following: { type: mongoose.Schema.Types.ObjectId, ref: "User", required: true },  
5     isActive: { type: Boolean, default: true }  
6   },  
7   { timestamps: true }  
8 );
```

Hình 8: Sơ đồ Follow Schema

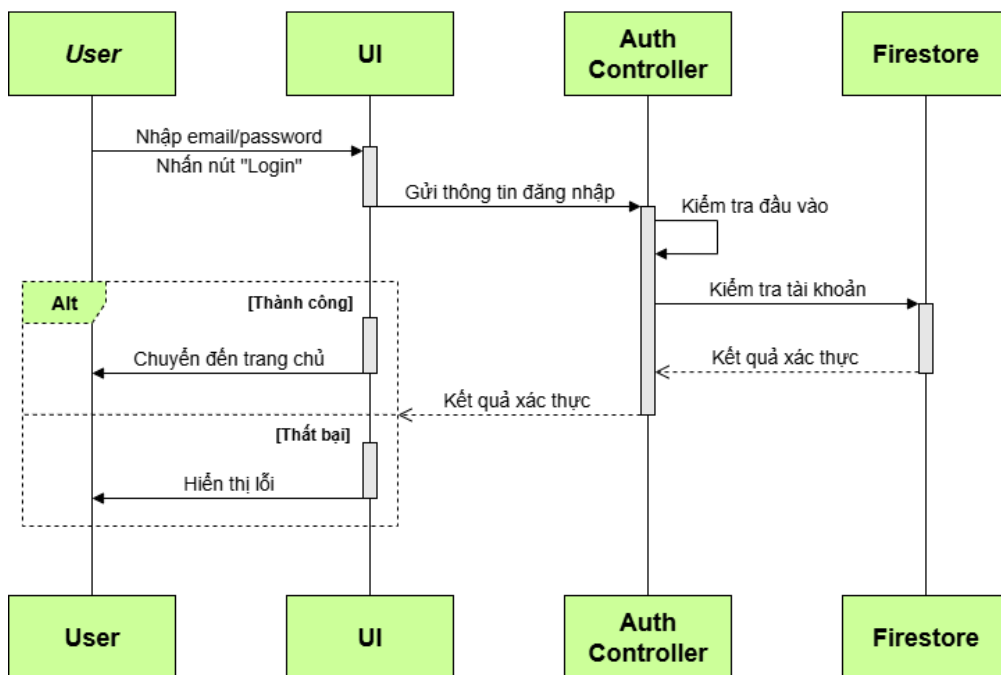
STT	Trường	Mô tả
1	follower	ID người theo dõi
2	following	ID người được theo dõi
3	isActive	Trạng thái hoạt động của mối quan hệ

6 Thiết kế hành vi và giao diện người dùng

6.1 Đăng nhập - đăng kí

6.1.1 Đăng nhập - Login

- Sequence Diagram:



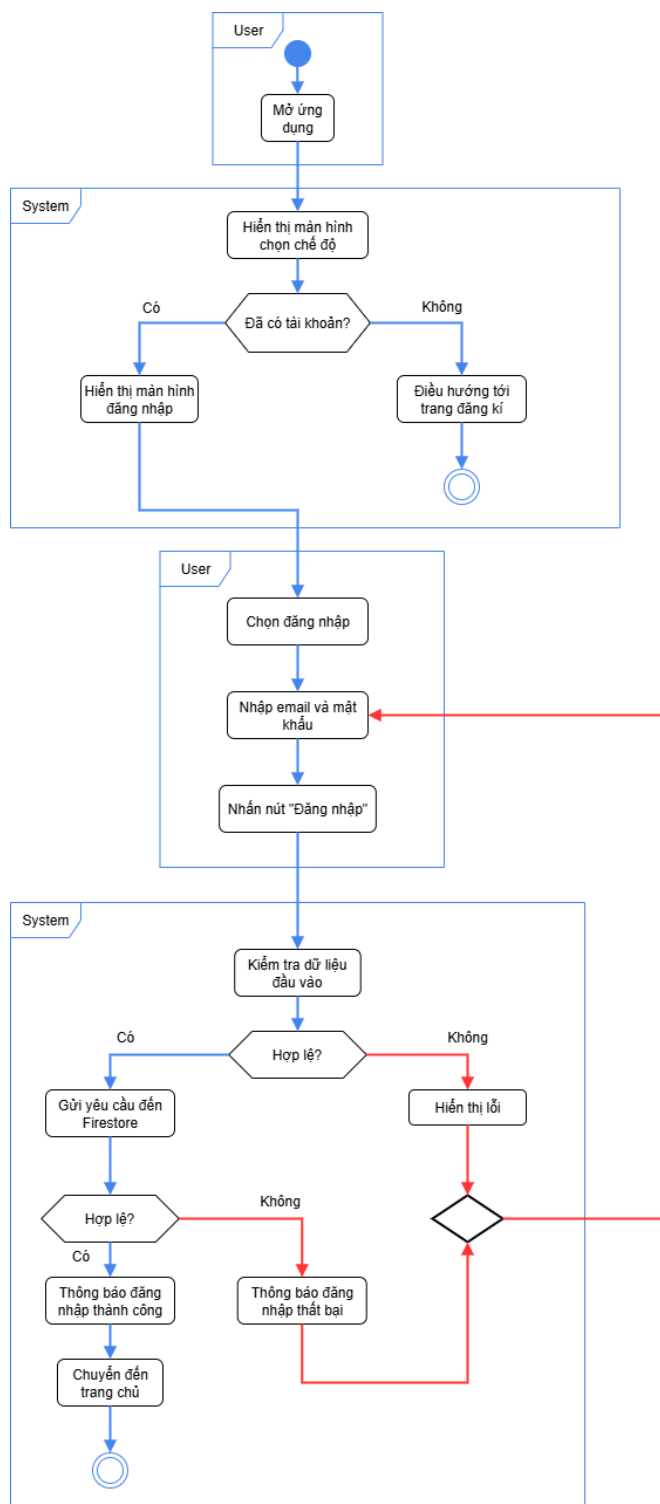
Hình 9: Sequence Diagram - Login

- **Mục đích:** Thể hiện quy trình đăng nhập của người dùng, bao gồm các bước từ khi người dùng nhập thông tin đến khi hệ thống xác thực và phản hồi kết quả.
- **Thành phần liên quan:**
 - Người dùng (User): người thực hiện thao tác đăng nhập.
 - Giao diện đăng nhập (Login UI): giao diện để người dùng nhập thông tin và hiển thị kết quả.
 - Hệ thống xác thực (Auth Controller): nhận thông tin đăng nhập và xử lý xác thực.
 - Cơ sở dữ liệu người dùng (Firestore): lưu trữ thông tin tài khoản.
- **Luồng tương tác:**
 1. Người dùng mở trang đăng nhập.
 2. Người dùng nhập email + mật khẩu và nhấn nút “Đăng nhập”.
 3. Giao diện đăng nhập gửi yêu cầu đăng nhập đến Hệ thống xác thực.
 4. Hệ thống xác thực kiểm tra dữ liệu đầu vào (có trống không, định dạng hợp lệ không).



5. Nếu hợp lệ, Hệ thống xác thực truy vấn Cơ sở dữ liệu người dùng để tìm tài khoản.
6. Cơ sở dữ liệu trả về thông tin tài khoản (nếu tồn tại).
7. Hệ thống xác thực phản hồi lại cho Giao diện đăng nhập:
 - Nếu đúng: thông báo đăng nhập thành công và chuyển đến giao diện trang chủ.
 - Nếu sai: thông báo lỗi (sai mật khẩu, tài khoản không tồn tại).

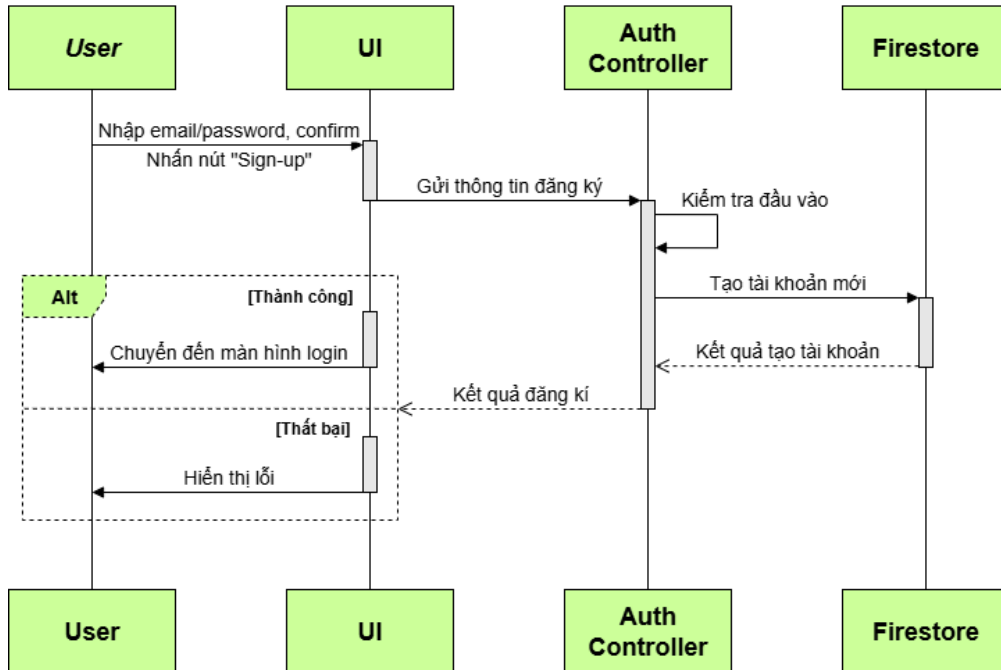
- **Activity Diagram:**



Hình 10: Activity Diagram - Login

6.1.2 Đăng ký - Register

- Sequence Diagram:



Hình 11: Sequence Diagram - Register

- Mục đích:** Thể hiện quy trình đăng ký tài khoản mới của người dùng, bao gồm các bước từ khi người dùng nhập thông tin đến khi hệ thống tạo tài khoản và phản hồi kết quả.

- Thành phần liên quan:**

- Người dùng (User): người thực hiện thao tác đăng ký.
- Giao diện đăng ký (Register UI): giao diện để người dùng nhập thông tin và hiển thị kết quả.
- Hệ thống xác thực (Auth Controller): nhận thông tin đăng ký và xử lý xác thực.
- Cơ sở dữ liệu người dùng (Firestore): lưu trữ thông tin tài khoản.

- Luồng tương tác:**

- Người dùng mở trang đăng ký.
- Người dùng nhập email, mật khẩu và xác nhận mật khẩu, sau đó nhấn nút “Đăng ký”.
- Giao diện đăng ký gửi yêu cầu tạo tài khoản đến Hệ thống xác thực.
- Hệ thống xác thực kiểm tra dữ liệu đầu vào (có trống không, mật khẩu có khớp không, định dạng email hợp lệ không).
- Nếu hợp lệ, Hệ thống xác thực truy vấn Cơ sở dữ liệu người dùng để kiểm tra xem tài khoản đã tồn tại hay chưa.

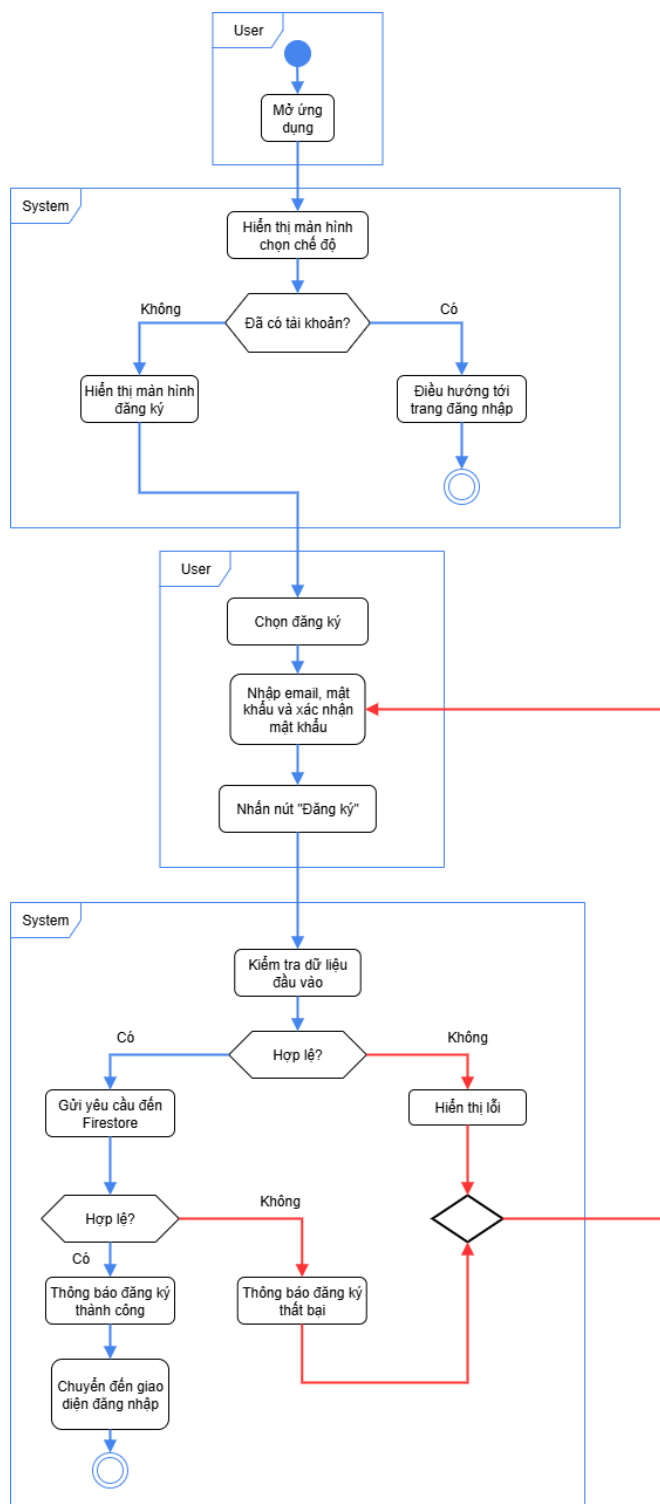


6. Cơ sở dữ liệu phản hồi kết quả kiểm tra.

7. Hệ thống xác thực phản hồi lại cho Giao diện đăng ký:

- Nếu tài khoản chưa tồn tại: tạo tài khoản mới, thông báo đăng ký thành công và điều hướng người dùng đến giao diện đăng nhập.
- Nếu tài khoản đã tồn tại hoặc dữ liệu không hợp lệ: thông báo lỗi (email đã tồn tại, mật khẩu không khớp, định dạng sai, v.v.).

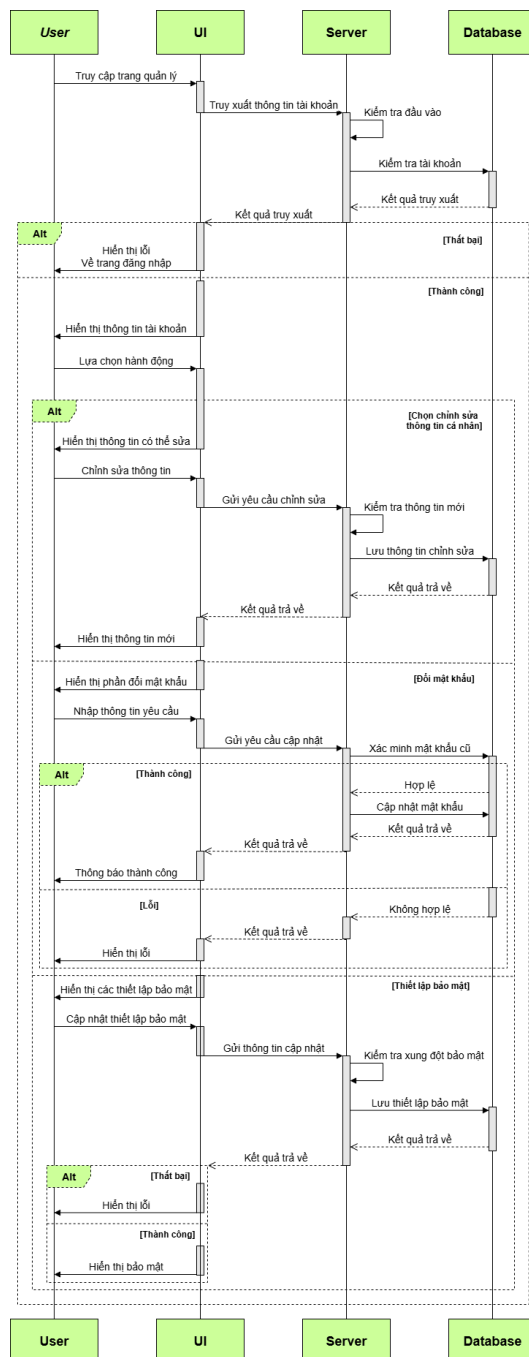
• **Activity Diagram:**



Hình 12: Activity Diagram - Register

6.2 Quản lý tài khoản - Account Management

- Sequence Diagram:



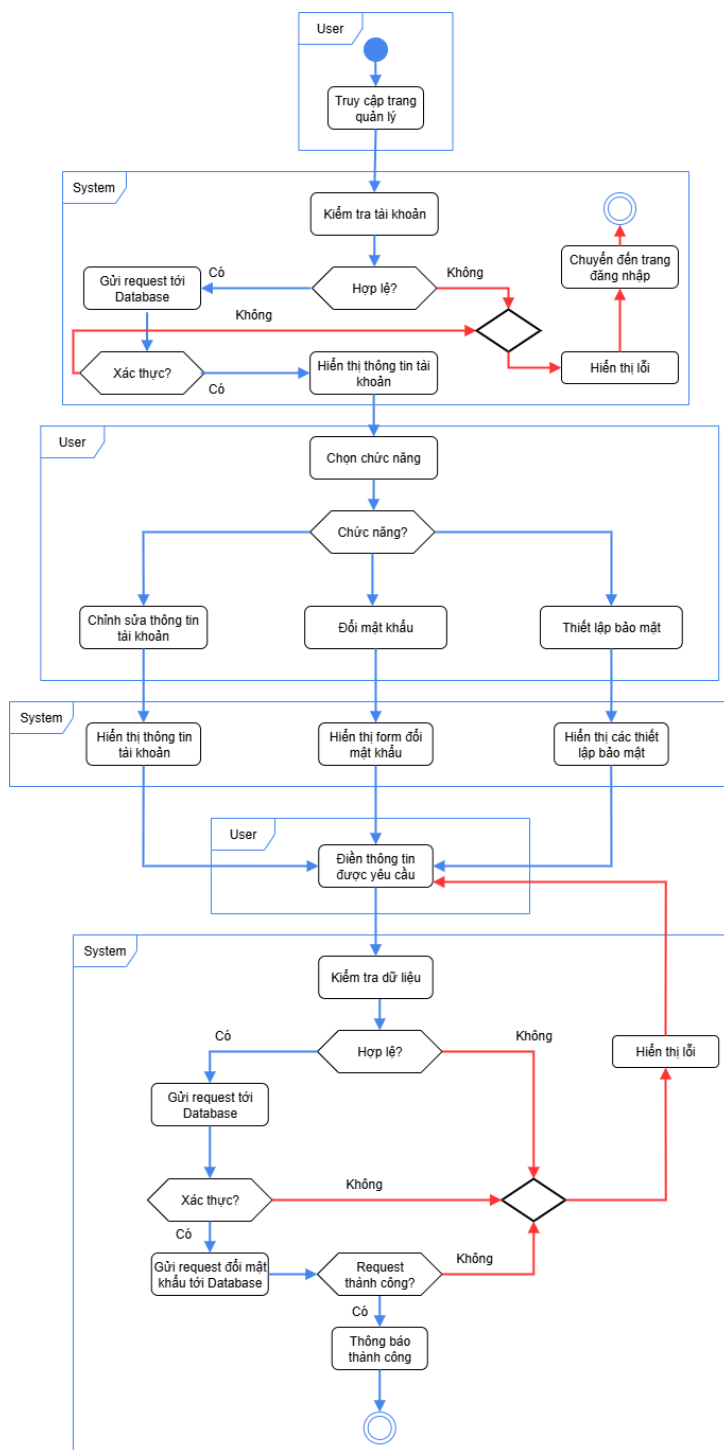
Hình 13: Sequence Diagram - Account Management

- **Mục đích:** Thể hiện các tùy chọn quản lý tài khoản cho người dùng.
- **Thành phần liên quan:**
 - Người dùng (User): người thực hiện thao tác đăng ký.
 - Giao diện quản lý (Management UI): giao diện để người dùng nhập thông tin và hiển thị kết quả.
 - Hệ thống xử lý (Server): nhận thông tin xác thực dữ liệu.
 - Cơ sở dữ liệu người dùng (Database): lưu trữ thông tin tài khoản.
- **Luồng tương tác:**
 1. Người dùng mở trang quản lý tài khoản.
 2. Hệ thống xử lý gửi yêu cầu đến cơ sở dữ liệu.
 3. Cơ sở dữ liệu phản hồi kết quả kiểm tra.
 4. Hệ thống kiểm tra trạng thái tài khoản, nếu chưa hợp lệ, thông báo lỗi và điều hướng về trang đăng nhập.
 5. Nếu hợp lệ, hệ thống trả về thông tin tài khoản và hiển thị lên giao diện.
 6. Người dùng lựa chọn chức năng:
 - Nếu người dùng chọn chỉnh sửa thông tin tài khoản:
 - * Giao diện hiển thị thông tin tài khoản có thể chỉnh sửa.
 - * Người dùng tiến hành chỉnh sửa và xác nhận.
 - * Hệ thống tiếp nhận yêu cầu sửa chữa và kiểm tra tính đúng đắn của dữ liệu.
 - * Nếu thông tin hợp lệ, hệ thống gửi yêu cầu cập nhật dữ liệu đến cơ sở dữ liệu.
 - * Cơ sở dữ liệu cập nhật dữ liệu mới và phản hồi nội dung cập nhật.
 - * Hệ thống gửi trả nội dung mới.
 - * Giao diện cập nhật theo nội dung mới của người dùng.
 - Nếu người dùng chọn đổi mật khẩu:
 - * Giao diện hiển thị biểu mẫu nhập mật khẩu cũ và mật khẩu mới.
 - * Người dùng nhập dữ liệu theo yêu cầu và xác nhận.
 - * Hệ thống tiếp nhận mật khẩu cũ và gửi đến cơ sở dữ liệu kiểm tra hợp lệ.
 - * Cơ sở dữ liệu phản hồi kết quả.
 - * Nếu dữ liệu hợp lệ, hệ thống gửi thông tin mật khẩu mới đến cơ sở dữ liệu.
 - * Cơ sở dữ liệu cập nhật mật khẩu mới và phản hồi nội dung cập nhật.
 - * Hệ thống gửi trả phản hồi thành công.
 - * Giao diện thông báo cập nhật thành công.
 - Nếu người dùng chọn thiết lập bảo mật:
 - * Giao diện hiển thị các thiết lập bảo mật.
 - * Người dùng xác nhận các thiết lập bảo mật.
 - * Hệ thống tiếp nhận và gửi yêu cầu đến cơ sở dữ liệu.
 - * Cơ sở dữ liệu cập nhật và phản hồi nội dung cập nhật.
 - * Hệ thống gửi trả phản hồi thành công.
 - * Giao diện cập nhật thiết lập mới nhất.



7. Nếu phản hồi là thất bại, giao diện hiển thị lỗi cho người dùng và khôi phục thiết lập ban đầu.

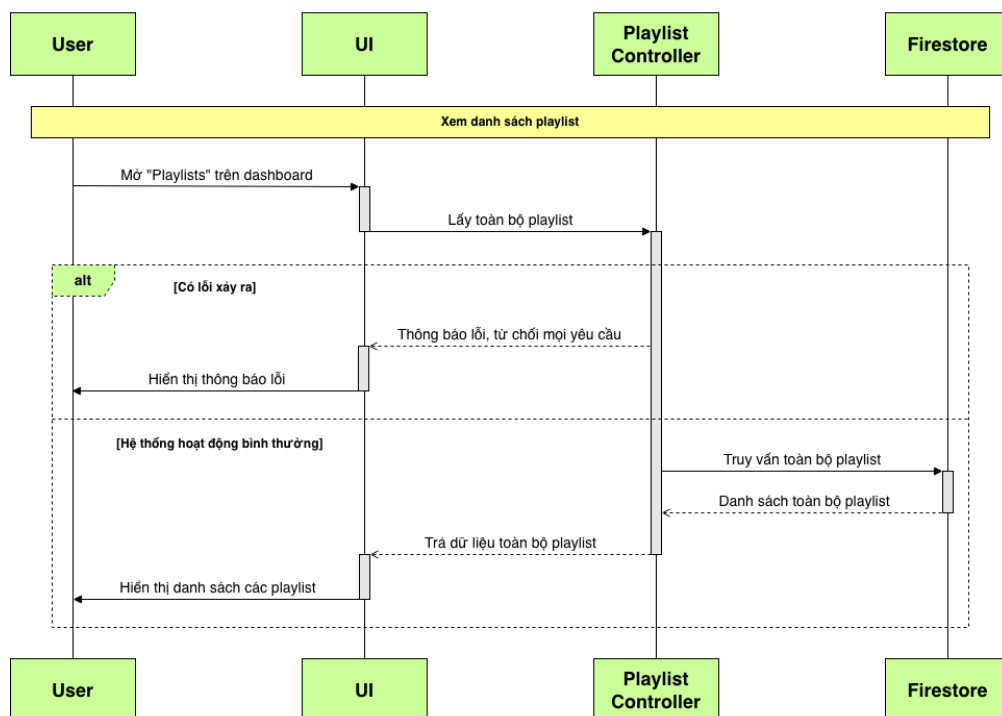
- **Activity Diagram:**



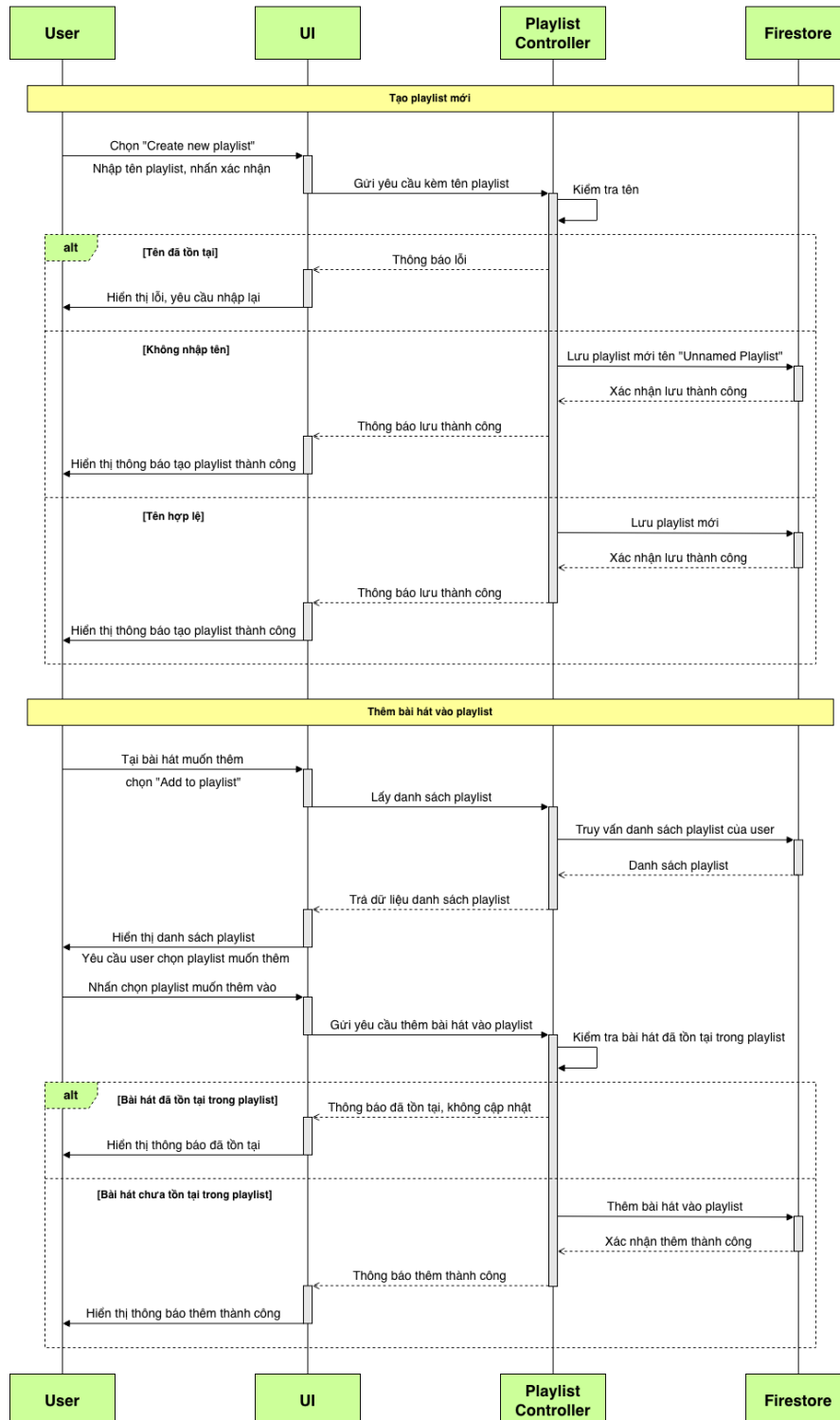
Hình 14: Activity Diagram - Account Management

6.3 Quản lý danh sách nhạc - Playlist Management

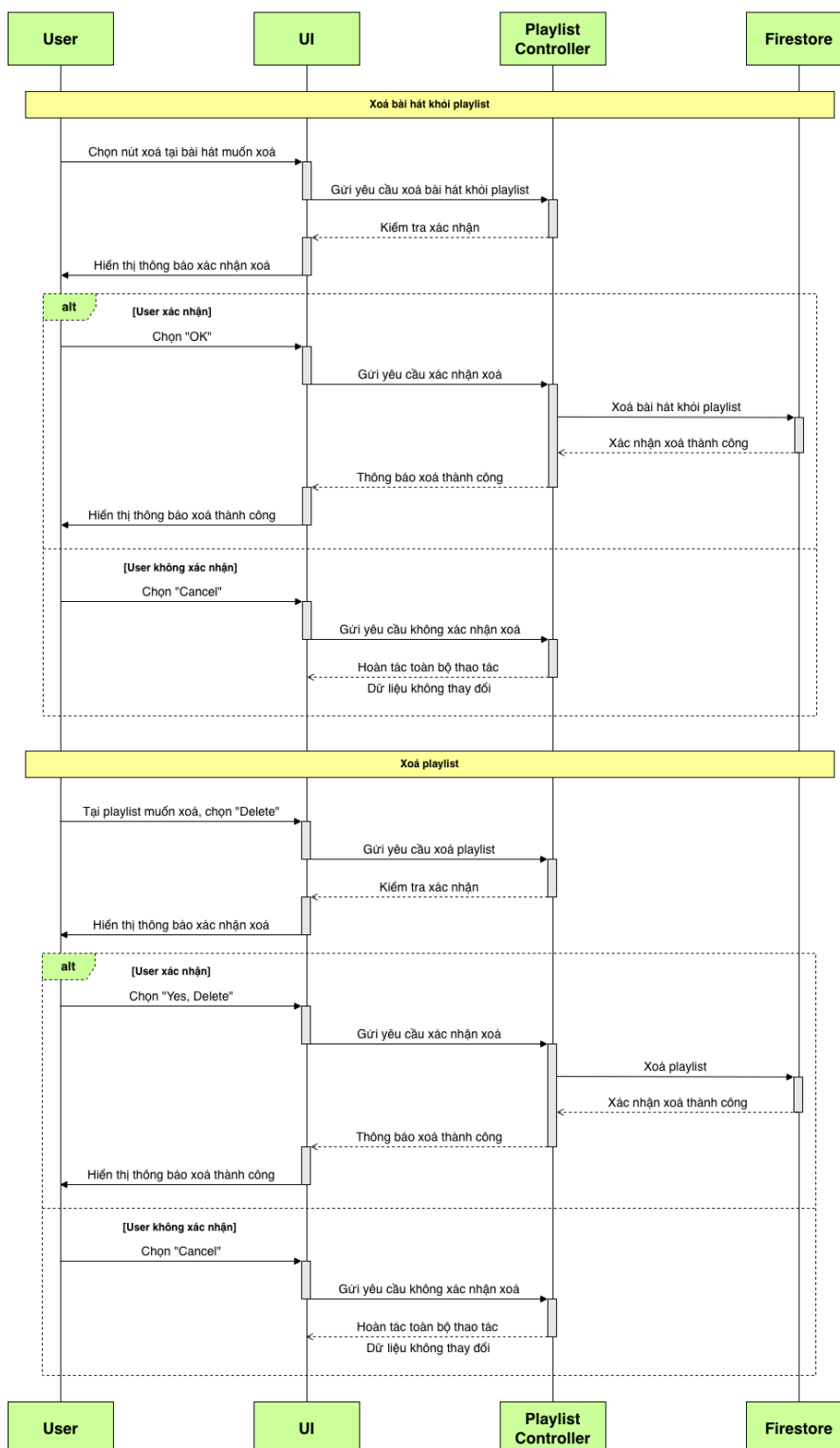
- Sequence Diagram:



Hình 15: Sequence Diagram - Playlist Management (Access "Playlists")



Hình 16: Sequence Diagram - Playlist Management (Create Playlist & Add Song To Playlist)



Hình 17: Sequence Diagram - Playlist Management
(Remove Song From Playlist & Delete Playlist)

- **Mục đích:** Thể hiện quy trình người dùng xem và thực hiện các thao tác điều khiển, quản lý danh sách nhạc (playlist) cá nhân của người dùng: truy cập "Playlists" trên dashboard, tạo playlist mới, thêm bài hát vào playlist, xóa bài hát khỏi playlist và xóa playlist.

- **Thành phần liên quan:**

- Người dùng (User): người thực hiện các thao tác quản lý playlist.
- Giao diện quản lý (Management UI): giao diện để người dùng thao tác, nhập thông tin và hiển thị thông báo/kết quả.
- Hệ thống xử lý (Playlist Controller): nhận thông tin và xử lý xác thực dữ liệu.
- Cơ sở dữ liệu người dùng (Firestore): lưu trữ thông tin playlist của tài khoản người dùng.

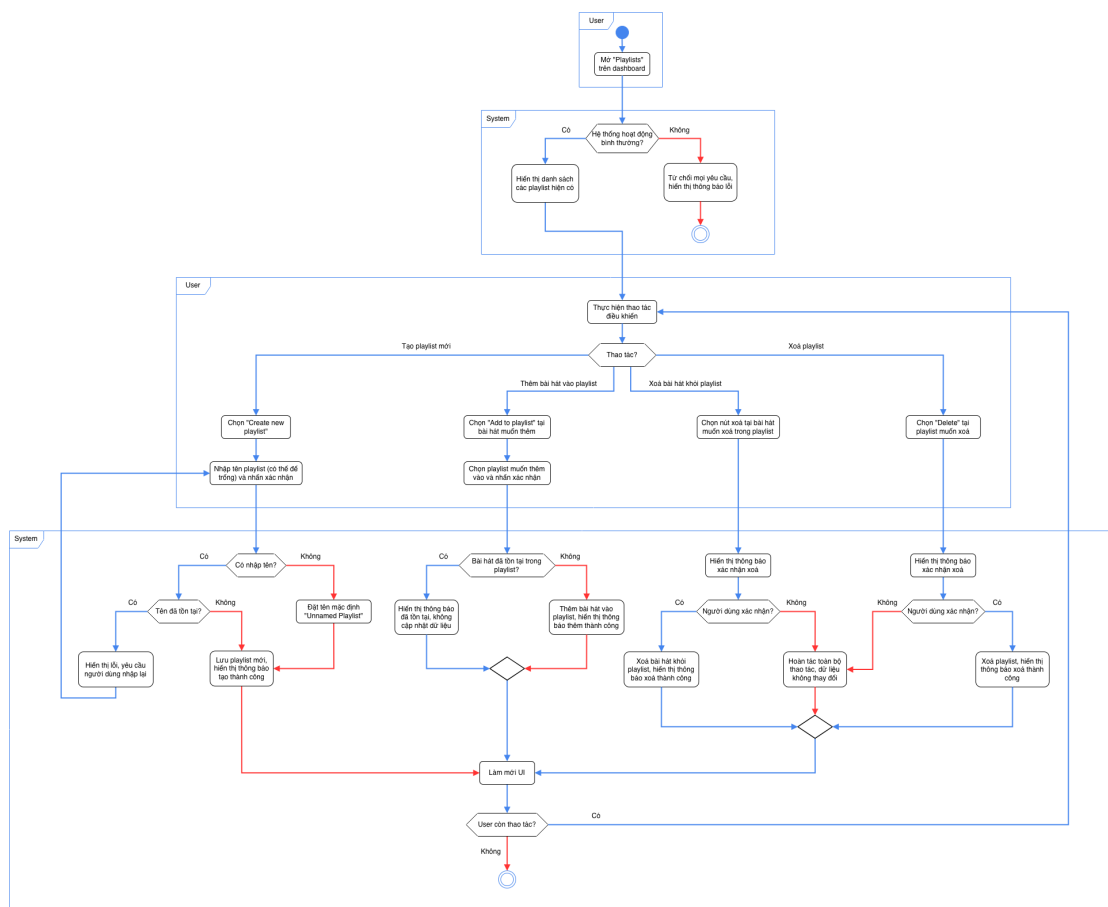
- **Luồng tương tác:**

1. Người dùng truy cập "Playlists" trên dashboard.
2. Nếu có lỗi xảy ra (mất kết nối máy chủ, hệ thống bận,...), hệ thống từ chối mọi yêu cầu từ người dùng và giao diện hiển thị thông báo lỗi.
3. Nếu hệ thống hoạt động bình thường, hệ thống xử lý gửi yêu cầu đến cơ sở dữ liệu, cơ sở dữ liệu phản hồi kết quả là danh sách nhạc của người dùng hiện tại và hiển thị lên giao diện.
4. Người dùng thực hiện thao tác điều khiển:
 - Nếu người dùng muốn tạo playlist mới:
 - * Người dùng chọn "Create new playlist" trên giao diện, nhập tên playlist (có thể để trống) và nhấn xác nhận.
 - * Hệ thống tiếp nhận yêu cầu và kiểm tra tên playlist.
 - * Nếu tên đã tồn tại, hệ thống thông báo lỗi và yêu cầu người dùng nhập lại.
 - * Nếu người dùng không nhập tên cho playlist mà trực tiếp xác nhận, hệ thống sẽ tự động đặt tên theo định dạng mặc định "Unnamed Playlist" và lưu vào cơ sở dữ liệu.
 - * Nếu tên hợp lệ, hệ thống gửi yêu cầu tạo playlist mới đến cơ sở dữ liệu, cơ sở dữ liệu lưu trữ playlist mới và phản hồi kết quả.
 - * Giao diện hiển thị thông báo tạo playlist mới thành công và cập nhật theo dữ liệu mới.
 - Nếu người dùng muốn thêm bài hát vào playlist:
 - * Người dùng chọn "Add to playlist" với bài hát muốn thêm.
 - * Hệ thống tiếp nhận yêu cầu và truy vấn đến cơ sở dữ liệu để hiển thị danh sách playlist hiện có để người dùng chọn.
 - * Người dùng chọn playlist muốn thêm bài hát vào và nhấn xác nhận.
 - * Hệ thống tiếp nhận yêu cầu và kiểm tra bài hát đã tồn tại trong playlist hay chưa.
 - * Nếu bài hát đã tồn tại trong playlist, hệ thống hiển thị thông báo đã tồn tại và không cập nhật bài hát.
 - * Nếu bài hát chưa tồn tại trong playlist, hệ thống gửi yêu cầu thêm bài hát vào playlist đến cơ sở dữ liệu.
 - * Cơ sở dữ liệu thêm bài hát vào playlist và phản hồi kết quả.

- * Giao diện hiển thị thông báo thêm thành công và cập nhật theo dữ liệu mới.
- Nếu người dùng muốn xoá bài hát khỏi playlist:
 - * Người dùng chọn nút xoá tại bài hát muốn xoá trong playlist.
 - * Hệ thống hiển thị thông báo xác nhận xoá bài hát.
 - * Nếu người dùng xác nhận xoá bài hát, hệ thống tiếp nhận và gửi yêu cầu đến cơ sở dữ liệu, cơ sở dữ liệu sẽ xoá bài hát khỏi playlist và phản hồi kết quả, giao diện hiển thị thông báo xoá thành công và cập nhật theo dữ liệu mới.
 - * Nếu người dùng không xác nhận xoá bài hát mà chọn "Cancel", hệ thống hoàn tác toàn bộ thao tác và cập nhật lại trạng thái ban đầu của danh sách playlist, dữ liệu không thay đổi.
- Nếu người dùng muốn xoá playlist:
 - * Người dùng chọn "Delete" tại playlist muốn xoá.
 - * Hệ thống hiển thị thông báo xác nhận xoá playlist.
 - * Nếu người dùng xác nhận xoá playlist, hệ thống tiếp nhận và gửi yêu cầu đến cơ sở dữ liệu, cơ sở dữ liệu sẽ xoá playlist và phản hồi kết quả, giao diện hiển thị thông báo xoá thành công và cập nhật theo dữ liệu mới.
 - * Nếu người dùng không xác nhận xoá playlist mà chọn "Cancel", hệ thống hoàn tác toàn bộ thao tác và cập nhật lại trạng thái ban đầu của danh sách playlist, dữ liệu không thay đổi.

5. Hệ thống ghi nhận, cập nhật danh sách playlist của người dùng và làm mới UI.

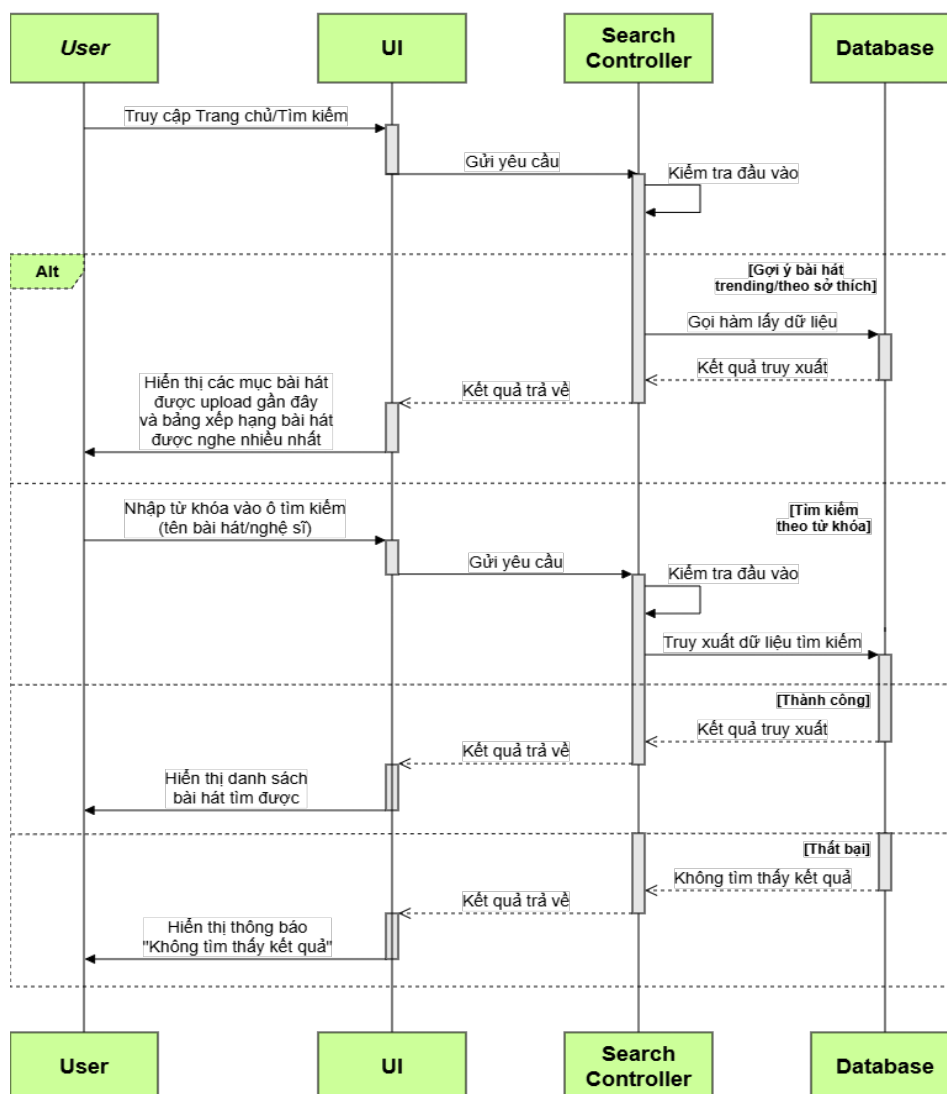
• **Activity Diagram:**



Hình 18: Activity Diagram - Playlist Management

6.4 Trang chủ và tìm kiếm - Homepage & Search

- Sequence Diagram:



Hình 19: Sequence Diagram - Discovery & Search

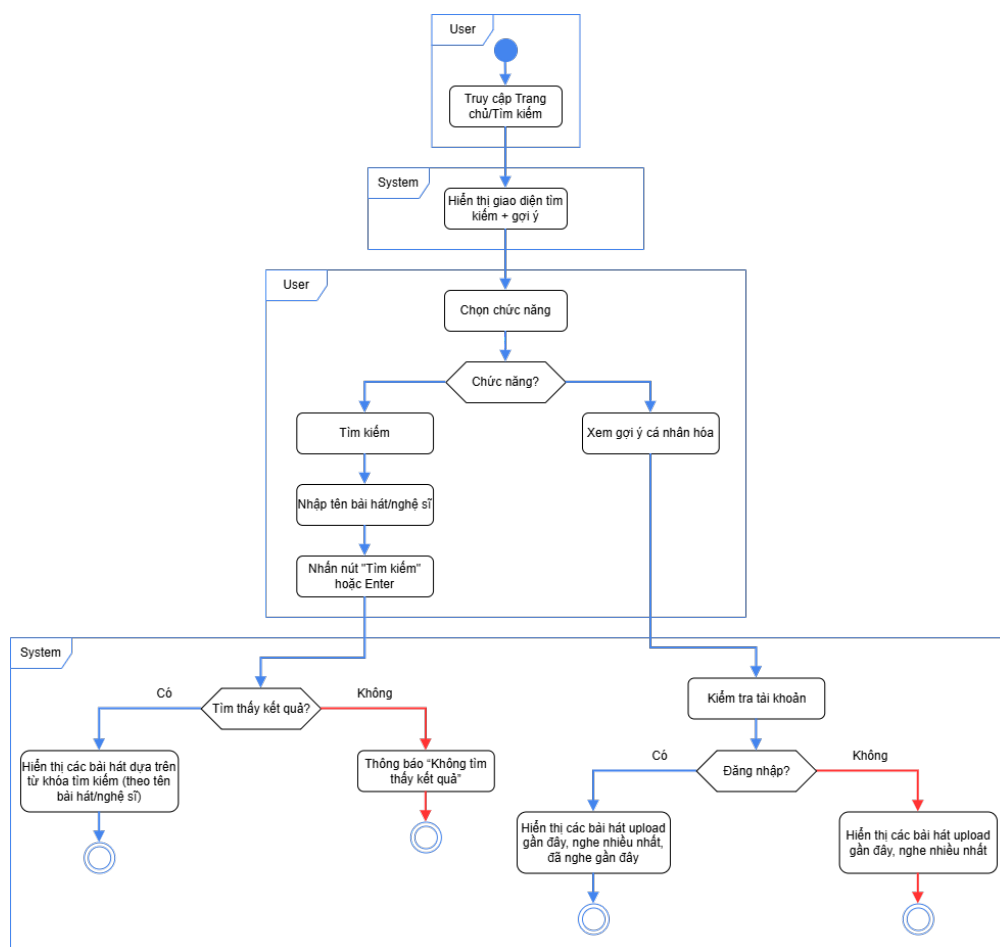
- **Mục đích:** Thể hiện quy trình người dùng truy cập tính năng Trang chủ/Tìm kiếm, bao gồm việc nhập từ khóa, nhận gợi ý cá nhân hóa, xem xu hướng (Trending/Top Chart), và kết quả trả về từ hệ thống.
- **Thành phần liên quan:**
 - Người dùng (User): thực hiện thao tác truy cập trang chủ/tìm kiếm, nhập từ khóa hoặc xem gợi ý.
 - Giao diện người dùng (UI): nơi hiển thị ô tìm kiếm, gợi ý, kết quả tìm kiếm.
 - Search Controller: xử lý yêu cầu tìm kiếm, kiểm tra dữ liệu đầu vào và gọi truy vấn dữ liệu.

- Database: lưu trữ dữ liệu bài hát, nghệ sĩ, playlist, lịch sử nghe để phục vụ tìm kiếm và gợi ý.

- **Luồng tương tác:**

1. Người dùng truy cập **Trang chủ/Tìm kiếm**.
2. UI gửi yêu cầu đến Search Controller để lấy dữ liệu.
3. Search Controller kiểm tra yêu cầu, gọi dữ liệu từ Database.
4. Database trả kết quả là danh sách các bài hát dựa theo tên bài hát/nghe sĩ.
5. Người dùng nhập từ khóa (tên bài hát/nghe sĩ).
6. UI gửi yêu cầu tìm kiếm đến Search Controller.
7. Search Controller kiểm tra dữ liệu, truy xuất Database.
8. Database trả về kết quả:
 - Nếu thành công → danh sách bài hát tìm thấy.
 - Nếu thất bại → trả về thông báo “Không tìm thấy kết quả”.
9. UI hiển thị kết quả hoặc thông báo lỗi cho người dùng.

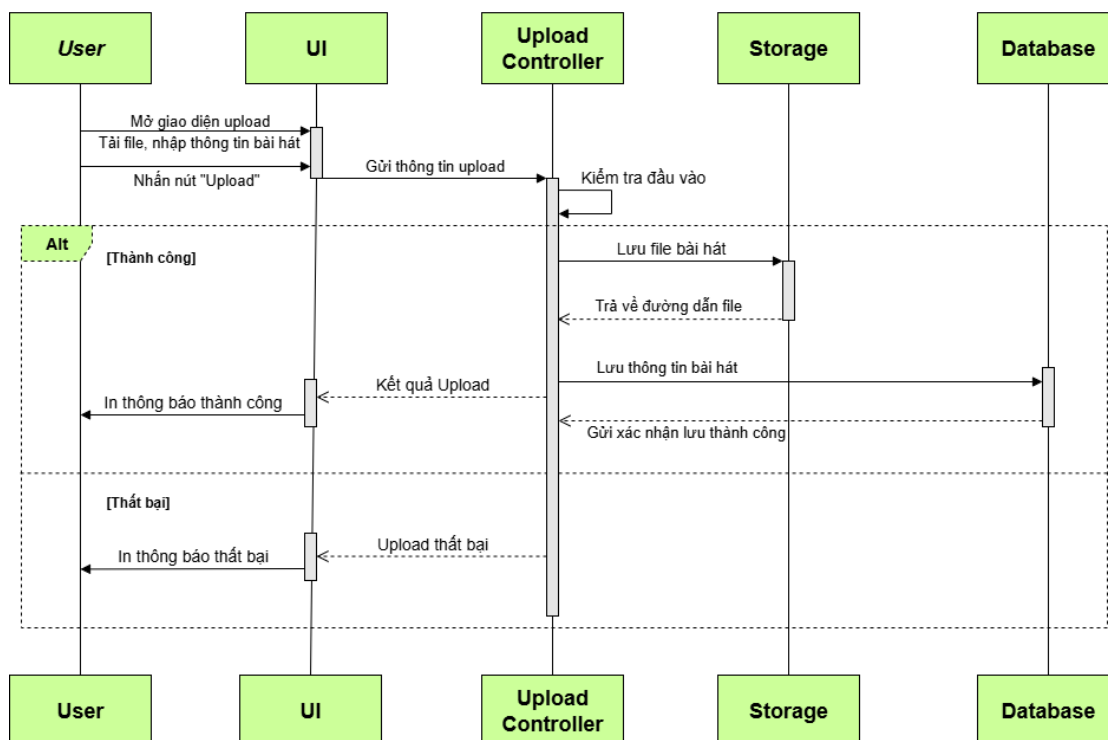
- **Activity Diagram:**



Hình 20: Activity Diagram - Discovery & Search

6.5 Upload bài hát - Song Upload

- Sequence Diagram:



Hình 21: Sequence Diagram - Upload Song

- **Mục đích:** Thể hiện quy trình người dùng truy cập tính năng Upload bài hát (bài hát + thumbnail + lyric (nếu có)), bao gồm việc chọn bài hát, nhập thông tin và nhận phản hồi từ hệ thống.

- **Thành phần liên quan:**

- Người dùng (User): thực hiện thao tác mở tab upload, chọn file và nhập thông tin bài hát.
- Giao diện người dùng (UI): giao diện để người dùng chọn file (file bài hát, thumbnail, lyric), nhập thông tin và hiển thị kết quả.
- Upload Controller: xử lý yêu cầu tải lên, kiểm tra dữ liệu đầu vào và gọi yêu cầu lưu dữ liệu.
- Storage: lưu trữ file bài hát.
- Database: lưu trữ dữ liệu bài hát (metadata: tên, ca sĩ, đường dẫn file,...).

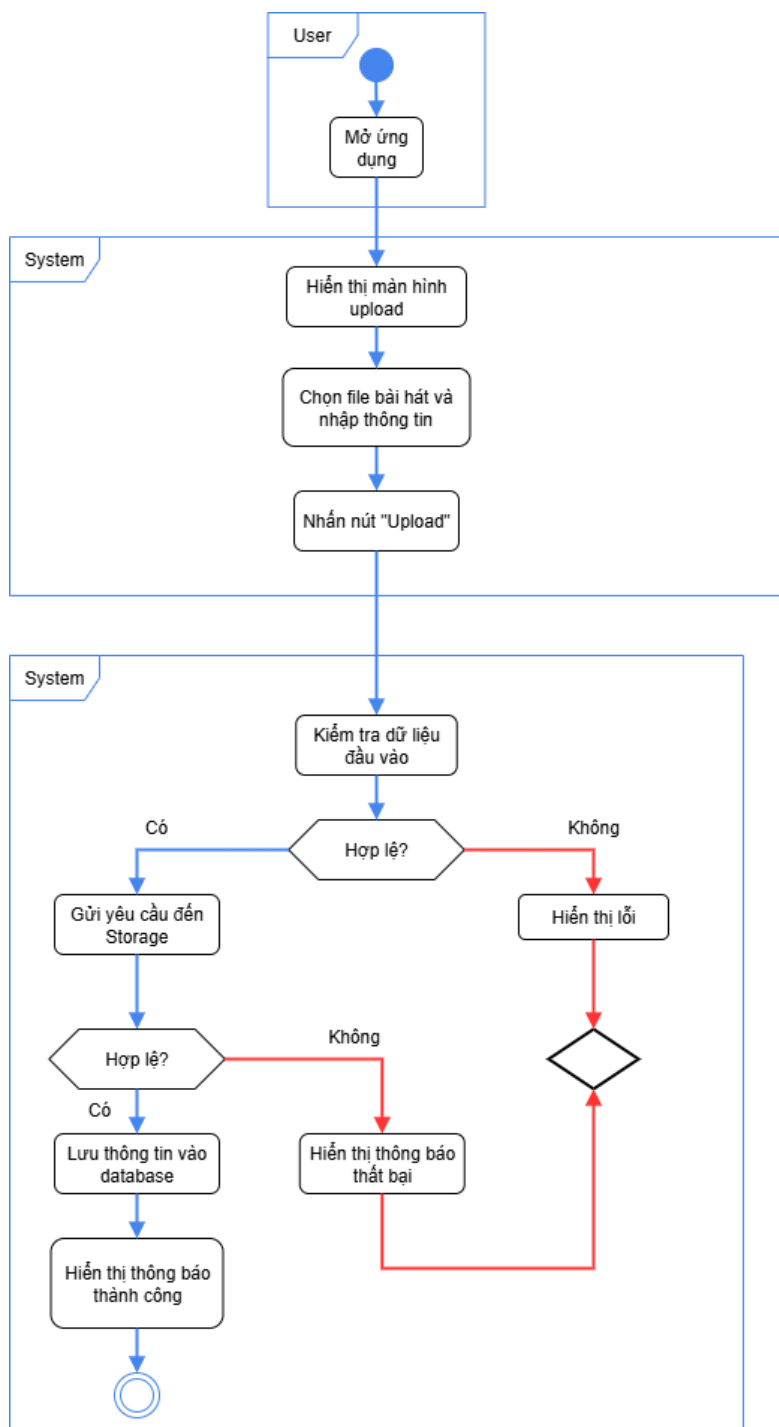
- **Luồng tương tác:**

1. Người dùng mở tab **Upload**.
2. Người dùng chọn file bài hát từ thiết bị và nhập thông tin (tên bài hát, nghệ sĩ, album, thể loại), chọn upload file ảnh thumbnail hoặc file lyric (nếu có).
3. UI gửi yêu cầu tải lên đến Upload Controller.
4. Upload Controller kiểm tra dữ liệu, gọi Storage để lưu file.



5. Storage lưu file và trả về đường dẫn.
6. Upload Controller lưu thông tin bài hát vào Database.
7. Database trả về kết quả (thành công/ thất bại).
8. Upload Controller phản hồi lại UI.
 - Nếu thành công → thông báo "Tải lên thành công".
 - Nếu thất bại → trả về thông báo lỗi (file không hợp lệ, lỗi kết nối,...).
9. UI hiển thị thông báo cho người dùng..

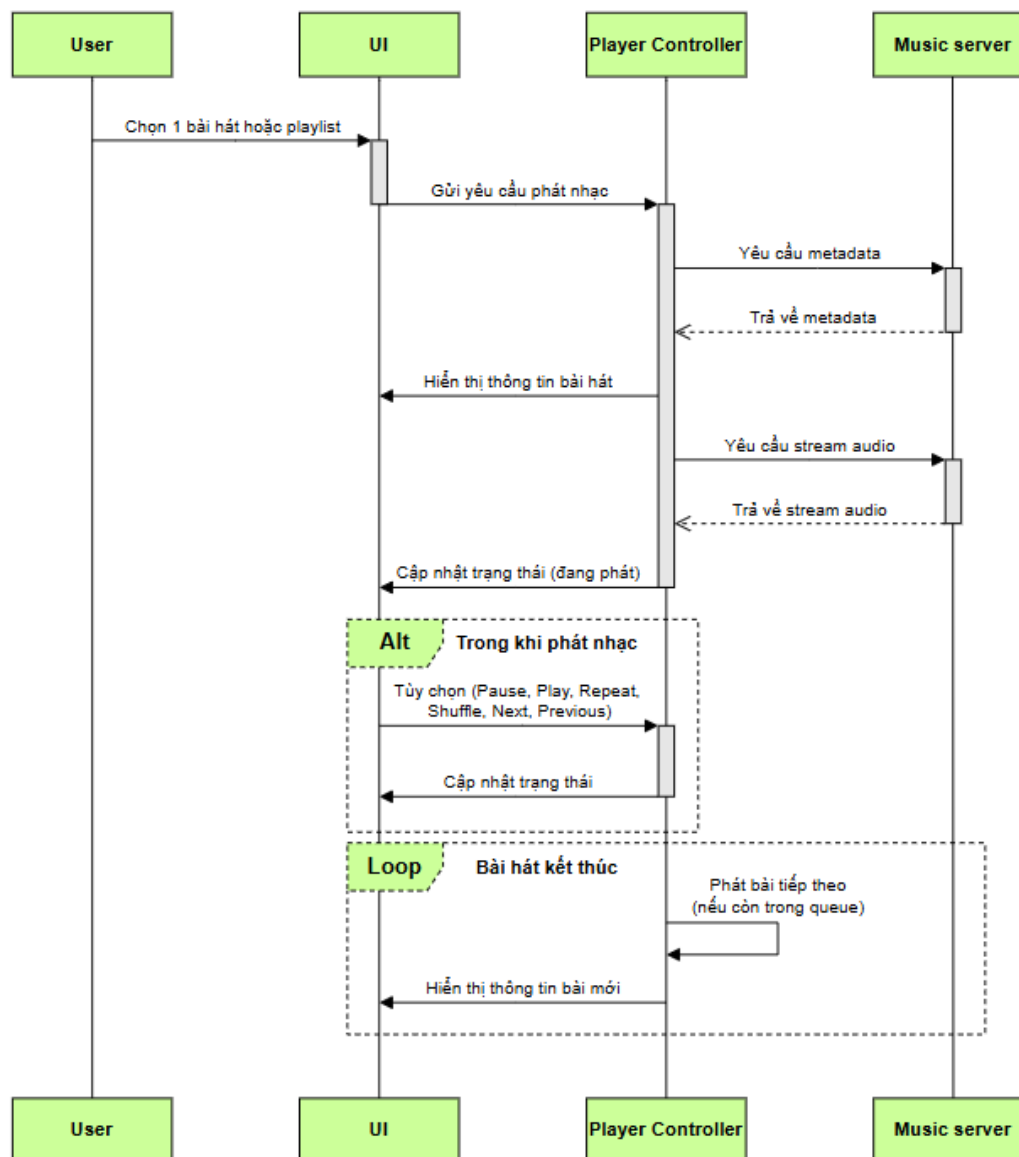
- **Activity Diagram:**



Hình 22: Activity Diagram - Upload Song

6.6 Phát nhạc - Player

- Sequence Diagram:



Hình 23: Sequence Diagram - Player

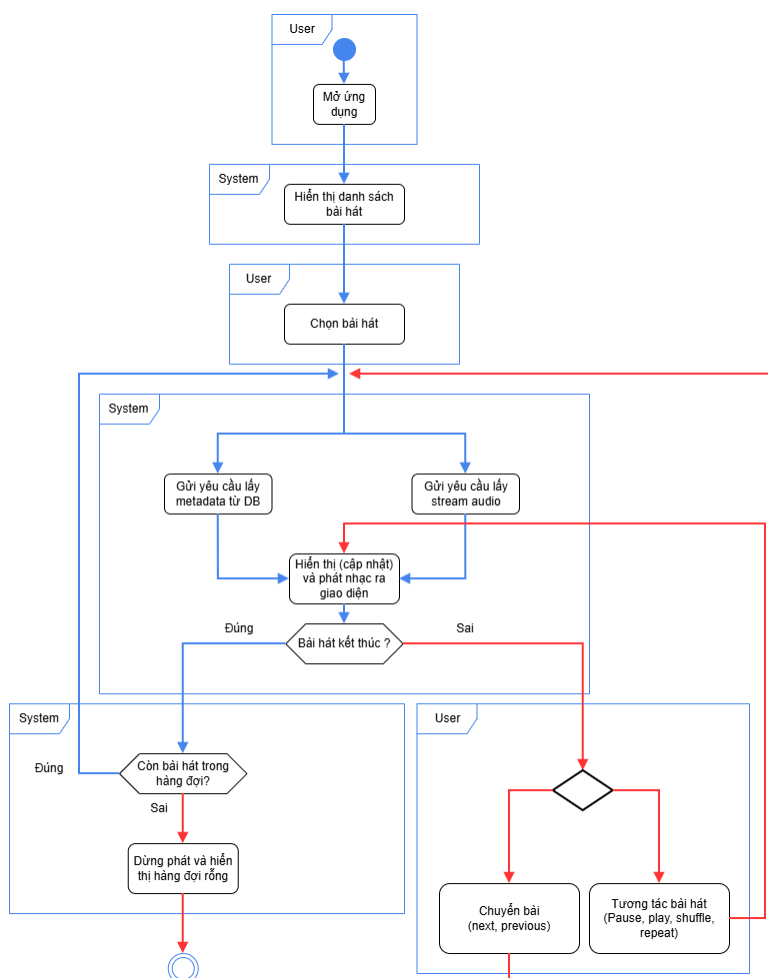
- **Mục đích:** Thể hiện quy trình người dùng thao tác với trình phát nhạc, từ việc phát 1 bài hát cho đến các tùy chọn như phát (play), tạm dừng (pause), chuyển bài (next), lùi bài (previous), trộn bài (shuffle), nghe lặp lại (repeat), tua đến đoạn khác.
- **Thành phần liên quan:**

- Người dùng (User): thực hiện thao tác trên thanh phát nhạc.
- Giao diện người dùng (UI): giao diện hiển thị thông tin bài hát gồm (tên bài hát, nghệ sĩ, ảnh thumbnail,...) và thanh phát nhạc với các nút bấm.
- Player Controller: xử lý yêu cầu lấy metadata và stream nhạc từ music server.
- Music server: hệ thống lưu trữ metagata của bài hát và các file dùng để stream nhạc.

• **Luồng tương tác:**

1. Người dùng chọn 1 bài hát.
2. UI gửi yêu cầu lên Player Controller.
3. Player Controller lấy metadata từ Music server và gửi yêu cầu stream nhạc.
4. Music server trả về metadata và stream audio.
5. UI cập nhật các thông tin của bài hát và cập nhật thanh thời lượng.
6. Người dùng tùy chọn các tính năng trên thanh phát nhạc.
7. Khi 1 bài kết thúc, hệ thống sẽ tự động phát bài kế tiếp dựa trên tính năng mà người dùng đã chọn.

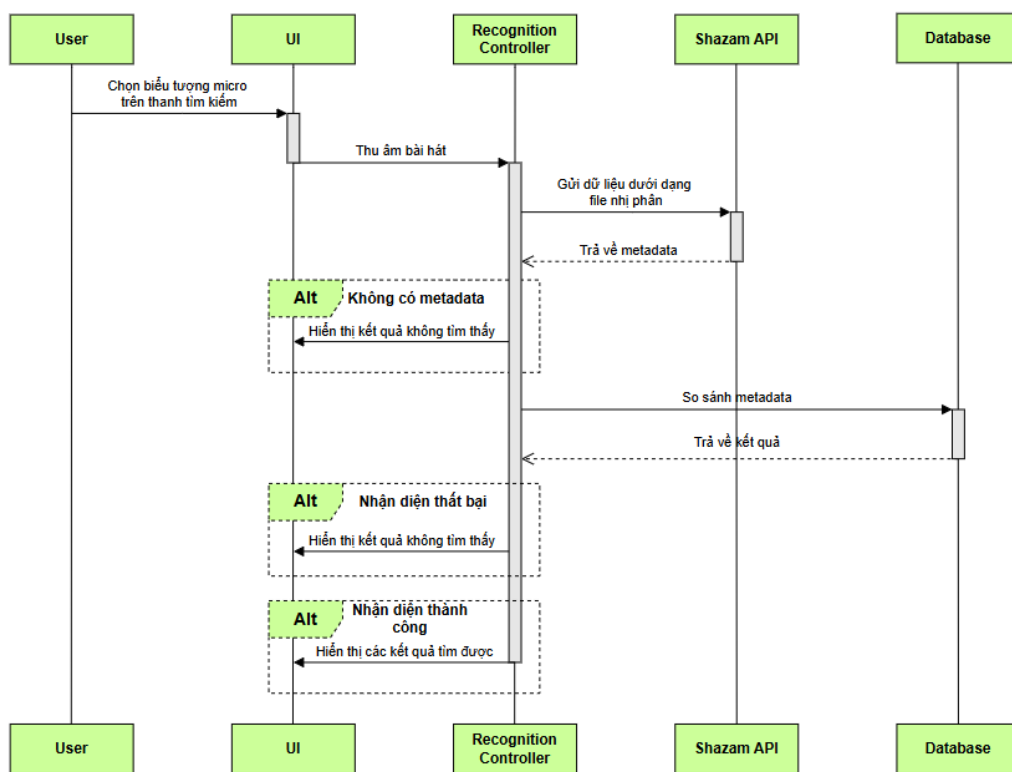
• **Activity Diagram:**



Hình 24: Activity Diagram - Player

6.7 Nhận diện bài hát - Recognition

- Sequence Diagram:



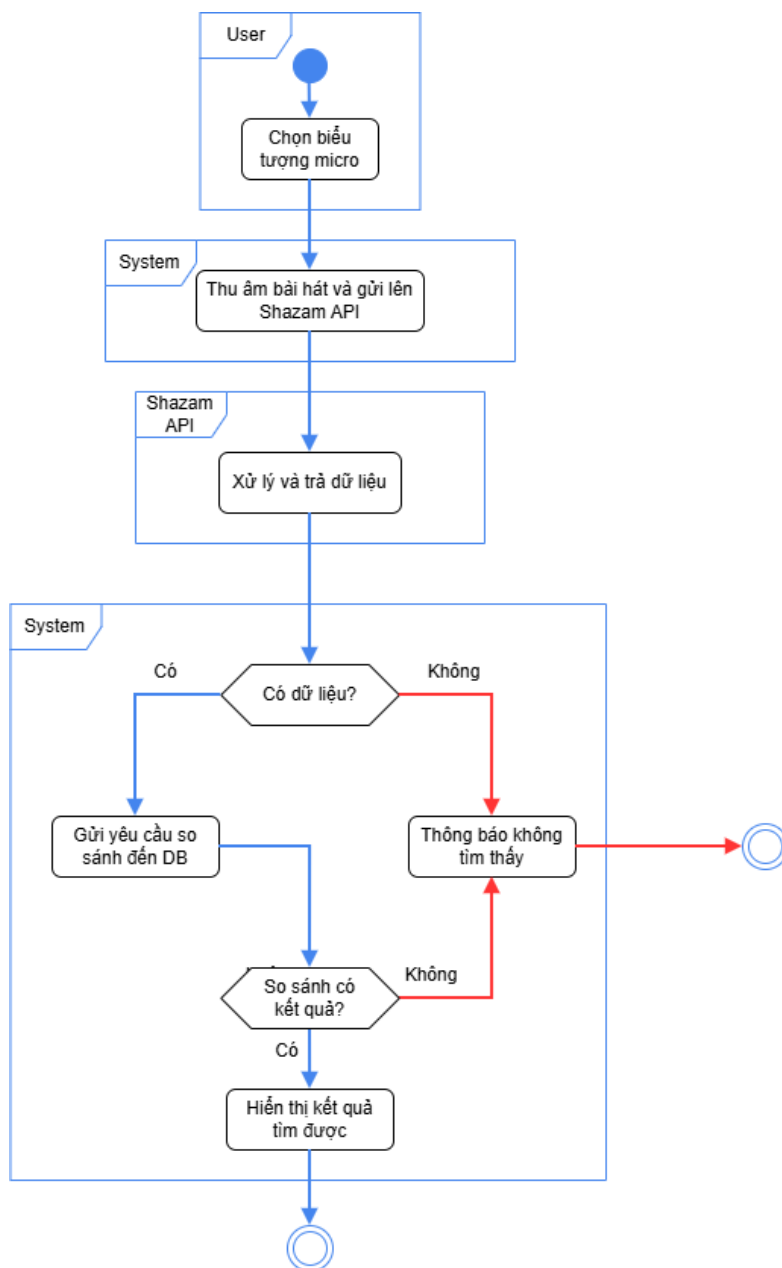
Hình 25: Sequence Diagram - Recognition

- **Mục đích:** Thể hiện quy trình người dùng thu âm bài hát từ micro của thiết bị để nhận diện bài hát từ đoạn thu âm đó và trả về kết quả dựa trên database của hệ thống.
- **Thành phần liên quan:**
 - Người dùng (User): thực hiện thao tác phát 1 bài hát từ loa ngoài.
 - Giao diện người dùng (UI): giao diện hiển thị trạng thái của tính năng nhận diện và kết quả tìm được.
 - Recognition Controller: xử lý file thu âm, và gửi yêu cầu đến Shazam api và so sánh kết quả với database.
 - Shazam api: api của Shazam cho phép nhận diện bài hát qua giai điệu.
 - Database: Lưu trữ toàn bộ dữ liệu bài hát của hệ thống để so khớp kết quả từ Shazam api.
- **Luồng tương tác:**
 1. Người dùng bấm vào biểu tượng micro trên thanh tìm kiếm.
 2. UI hiển thị giao diện nhận diện.
 3. Người dùng bấm bắt đầu.



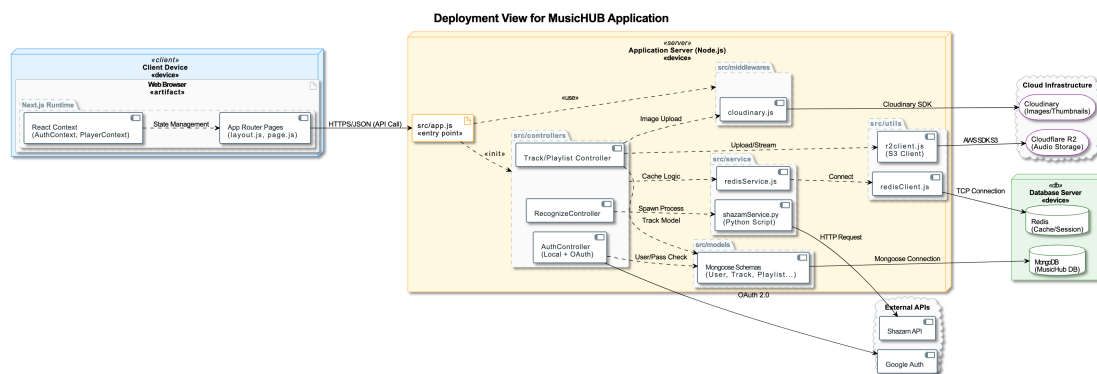
4. UI tiến hành ghi nhận âm thanh từ micro của thiết bị và gửi sang Recognition Controller.
5. Recognition controller xử lý file ghi âm thành file nhị phân và gửi yêu cầu đến Shazam api.
6. Shazam api trả về metadata.
7. Recognition controller so khớp metadata trong database và trả về kết quả.
8. UI hiển thị kết quả được trả về từ Recognition controller và người dùng có thể nhấn phát ngay nếu nhận diện thành công.

- **Activity Diagram:**



Hình 26: Activity Diagram - Recognition

7 Kiến trúc triển khai (Deployment View)



Hình 27: Deployment View của hệ thống

7.1 Tổng quan

Deployment View (Sơ đồ triển khai) mô tả kiến trúc vật lý của hệ thống MusicHUB, minh họa cách các thành phần phần mềm (Software Artifacts) được phân bố trên các node phần cứng hoặc môi trường ảo hóa. Sơ đồ thể hiện sự tương tác giữa Client, Application Server, Database Server và các dịch vụ Cloud bên thứ ba.

Để tăng tính trực quan, các thành phần trong sơ đồ được phân loại theo nhóm chức năng:

- **Client Tier (Màu xanh dương):** Giao diện tương tác người dùng.
- **Application Tier (Màu vàng):** Trung tâm xử lý logic nghiệp vụ.
- **Data Tier (Màu xanh lá):** Lưu trữ và quản lý dữ liệu.
- **Cloud & External (Màu xám):** Hạ tầng mạng và dịch vụ tích hợp.

7.2 Chi tiết các thành phần

7.2.1 Client Tier (Tầng Khách hàng)

Môi trường tương tác trực tiếp với người dùng cuối:

- **Thiết bị:** Web Browser (Trình duyệt web).
- **Artifacts:**
 - **Next.js Runtime:** Chạy ứng dụng Frontend (React 19) với Server-side Rendering.
 - **App Router Pages:** Hệ thống routing hiện đại của Next.js (layout.js, page.js).
 - **State Management:** Sử dụng React Context (AuthContext) để quản lý phiên đăng nhập cho cả hai phương thức (Local & OAuth).
- **Giao thức:** Giao tiếp qua HTTPS/JSON API.

7.2.2 Application Tier (Tầng Ứng dụng)

Đóng vai trò trung tâm xử lý nghiệp vụ, được triển khai trên môi trường **Node.js Server**.

- **Entry Point (src/app.js):** Điểm khởi chạy ứng dụng.
- **Controllers (src/controllers):**
 - **AuthController:** Quản lý hệ thống xác thực kép (Dual Authentication System):
 - * Xác thực truyền thống: Email/Password kết hợp mã hóa Bcrypt.
 - * Xác thực xã hội: Google OAuth 2.0.
 - **Track/Playlist Controller:** Quản lý streaming và danh sách phát.
 - **RecognizeController:** Xử lý nhận diện nhạc.
- **Integration Services:**
 - **Shazam Service:** Script Python (`shazamService.py`) để giao tiếp với Shazam API.
 - **Cloud Clients:** Các module kết nối Cloudflare R2 và Redis.

7.2.3 Data Tier (Tầng Dữ liệu)

- **MongoDB (Primary Database):**
 - Lưu trữ thông tin người dùng bao gồm mật khẩu đã mã hóa (Hashed Password) và thông tin Profile từ Google.
 - Sử dụng Mongoose Schemas để định nghĩa cấu trúc dữ liệu.
- **Redis (Cache & Session):** Lưu trữ Session đăng nhập (JWT Whitelist/Blacklist) để tối ưu hiệu năng xác thực.

7.3 Các luồng dữ liệu chính (Data Flows)

7.3.1 Quy trình Xác thực (Authentication Flow)

Hệ thống hỗ trợ hai luồng đăng nhập song song:

1. **Đăng nhập truyền thống (Traditional Login):** Client → AuthController → Kiểm tra Email → So khớp Bcrypt Hash → Tạo JWT Token → Lưu Session vào Redis.
2. **Đăng nhập qua Google (OAuth Login):** Client → AuthController → Redirect Google OAuth → Lấy Profile Data → Tạo/Cập nhật User → Tạo JWT Token → Lưu Session vào Redis.

7.3.2 Các luồng nghiệp vụ khác

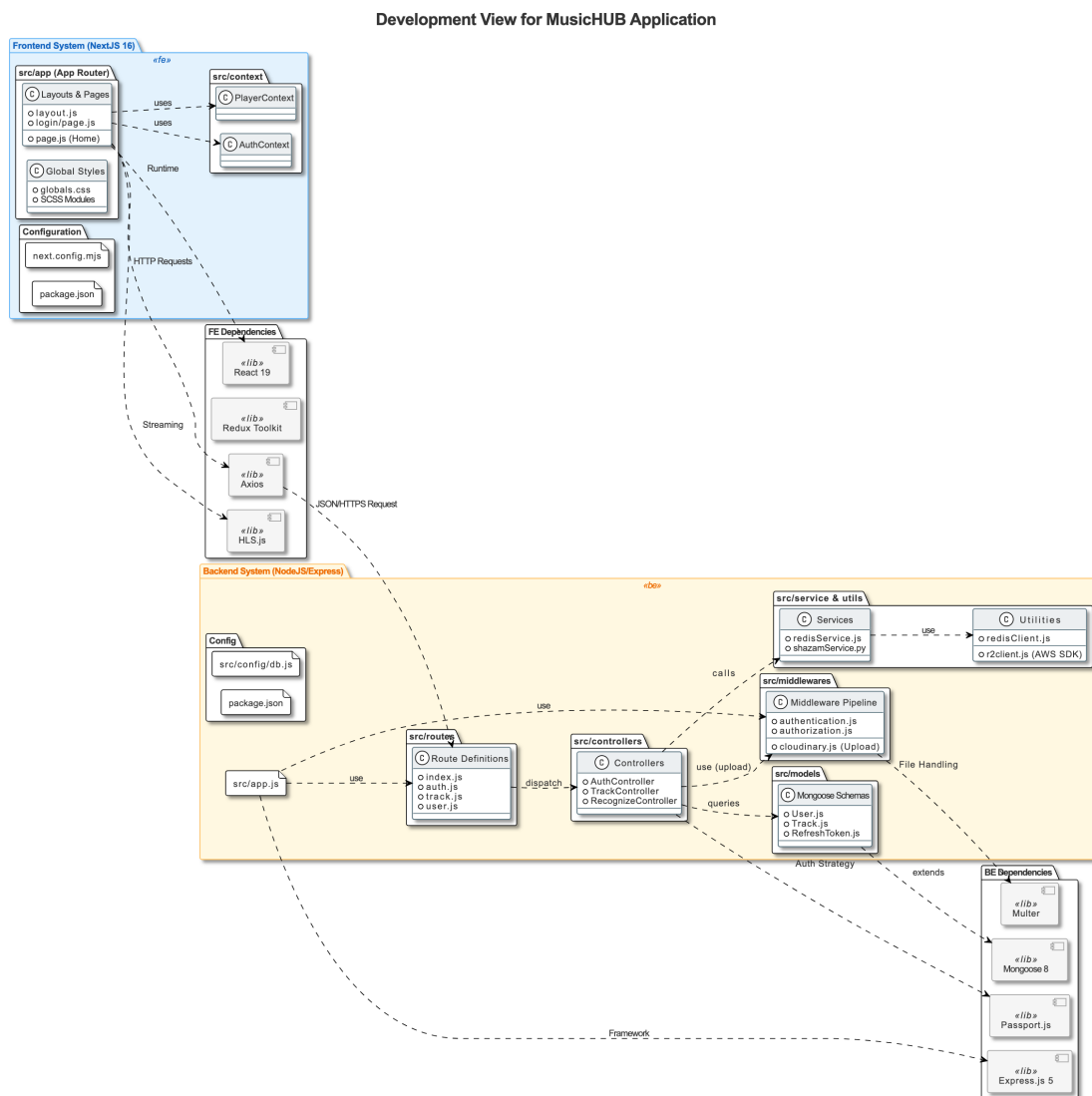
1. **Upload nhạc:** Client → Middleware Upload → Cloudinary (Ảnh) + R2 (Audio) → Lưu Metadata vào MongoDB.
2. **Streaming Nhạc:** Client → TrackController → Lấy URL từ Cloudflare R2 → Stream về Client.
3. **Nhận diện nhạc:** Client → RecognizeController → `shazamService.py` → Shazam API.



7.4 Đặc điểm kỹ thuật & Bảo mật

- **Password Security:** Mật khẩu người dùng không bao giờ được lưu dưới dạng văn bản thuần (plain text). Hệ thống sử dụng thư viện **Bcrypt** để băm (hash) mật khẩu trước khi lưu vào MongoDB.
- **Stateless Authentication:** Sử dụng **JWT (JSON Web Token)** để quản lý phiên làm việc, cho phép xác thực không trạng thái và dễ dàng mở rộng (scale).
- **Secure Communication:** Toàn bộ giao tiếp giữa Client và Server được mã hóa qua giao thức HTTPS/TLS.
- **Environment Security:** Các thông tin nhạy cảm (JWT Secret, Database URI, API Keys) được quản lý chặt chẽ qua biến môi trường.

8 Kiến trúc phát triển (Development View)



Hình 28: Development View của hệ thống

8.1 Tổng quan

Development View mô tả cách tổ chức mã nguồn, cấu trúc module và quản lý sự phụ thuộc (Dependencies) của hệ thống MusicHUB. Hệ thống được phát triển theo mô hình tách biệt hoàn toàn (Decoupled Architecture) giữa Frontend và Backend, cho phép hai phía được phát triển độc lập.

8.2 Tổ chức mã nguồn (Source Code Organization)

Hệ thống được chia thành hai thư mục gốc (monorepo-style structure):

- **fe/**: Chứa toàn bộ mã nguồn Frontend (Next.js).
- **be/**: Chứa toàn bộ mã nguồn Backend (Node.js).

8.2.1 Phân hệ Frontend (fe/)

Được xây dựng dựa trên **Next.js 16** và **React 19**, cấu trúc mã nguồn tuân theo kiến trúc *App Router* hiện đại:

- **src/app/**: Chứa các trang (Pages) và Layout. Đây là tầng giao diện (Presentation Layer), nơi định nghĩa các đường dẫn (Routes) và cấu trúc UI.
- **src/context/**: Quản lý trạng thái toàn cục (Global State) sử dụng React Context API.
 - **AuthContext.js**: Quản lý phiên đăng nhập, token người dùng.
 - **PlayerContext.js**: Quản lý trình phát nhạc (Play, Pause, Seek, Playlist queue).
- **Dependencies**:
 - Giao diện: SCSS Modules, FontAwesome, React Icons.
 - Xử lý Media: **hls.js** (Streaming audio), **react-virtuoso** (Danh sách ảo hóa hiệu năng cao).
 - Giao tiếp: **Axios** (HTTP Client).

8.2.2 Phân hệ Backend (be/)

Được xây dựng dựa trên **Express.js 5**, tổ chức theo mô hình **MVC (Model-View-Controller)** kết hợp với kiến trúc phân lớp (Layered Architecture):

- **src/app.js**: Điểm khởi chạy (Entry Point), nạp các cấu hình và middleware toàn cục.
- **src/routes/**: Định nghĩa các API Endpoint và điều hướng request đến Controller tương ứng.
- **src/controllers/**: Tầng xử lý nghiệp vụ (Business Logic).
 - **AuthController**: Xử lý logic đăng nhập (Local & OAuth).
 - **TrackController**: Xử lý logic upload và streaming nhạc.
 - **RecognizeController**: Điều phối tiến trình nhận diện nhạc.
- **src/models/**: Tầng truy cập dữ liệu (Data Access Layer), định nghĩa các Schema Mongoose (**User**, **Track**, **Playlist**).
- **src/middlewares/**: Các bộ lọc xử lý request pipeline (Authentication, Authorization, File Upload qua Cloudinary).
- **src/service/**: Các dịch vụ tích hợp đặc thù.
 - **shazamService.py**: Script Python tích hợp để xử lý tín hiệu âm thanh.
 - **redisService.js**: Service tương tác với Redis Cache.

8.3 Công nghệ sử dụng (Technology Stack)

Phân hệ	Hạng mục	Công nghệ / Thư viện
Frontend	Core Framework	Next.js 16, React 19
	State Management	React Context, Redux Toolkit
	Media	HLS.js
	Styling	SCSS, CSS Modules
Backend	Runtime	Node.js, Express.js 5
	Database	Mongoose (MongoDB ODM), IORedis
	Authentication	Passport.js, JWT, Bcrypt
	File Processing	Multer, Cloudinary SDK, AWS S3 SDK

Bảng 20: Danh sách các công nghệ và thư viện chính

8.4 Quy trình phát triển (Development Workflow)

- Môi trường:** Sử dụng biến môi trường (.env) để quản lý cấu hình nhạy cảm (DB Connection, API Keys) riêng biệt cho Development và Production.
- Frontend Dev:** Chạy trên port 3000 với tính năng Hot Module Replacement (HMR) của Next.js.
- Backend Dev:** Chạy trên port 8080 sử dụng nodemon để tự động khởi động lại server khi thay đổi code.
- Inter-communication:** Frontend giao tiếp với Backend thông qua RESTful API, sử dụng JWT Bearer Token trong Header để xác thực.

9 Advanced section

9.1 Cloud Deployment and Service (Triển khai và dịch vụ đám mây)

Hệ thống được triển khai theo mô hình tách lớp: Frontend chạy trên Vercel và Backend chạy trên Railway. Vercel hỗ trợ triển khai ứng dụng web với CDN toàn cầu và khả năng mở rộng tự động, trong khi Railway cung cấp môi trường chạy dịch vụ backend dạng container, dễ quản lý biến môi trường và theo dõi log. Việc sử dụng hai nền tảng cloud giúp hệ thống đáp ứng tốt nhu cầu mở rộng và giảm chi phí vận hành hạ tầng.

- Cloud Platform Setup:** Thay vì sử dụng AWS, Google Cloud hay Azure, nhóm lựa chọn Vercel và Railway vì phù hợp quy mô dự án và hỗ trợ tự động build-deploy trực tiếp từ Git. Cả hai nền tảng đều đảm bảo khả năng scale linh hoạt mà không cần cấu hình máy chủ thủ công.
- Database Management:** Cơ sở dữ liệu MongoDB được triển khai trên môi trường cloud của Railway, hỗ trợ backup và snapshot tự động. Nhóm đã kiểm tra quá trình khôi phục dữ liệu từ một bản snapshot nhằm bảo đảm an toàn khi xảy ra sự cố hệ thống.
- Continuous Deployment:** CI/CD được thực hiện thông qua GitHub tích hợp với Vercel và Railway. Khi có commit mới lên repository, Vercel tự động build và triển khai frontend, trong khi Railway cập nhật backend lên môi trường production. Quy trình này đảm bảo continuous delivery mà không cần thao tác thủ công, giúp nâng cấp nhanh và ổn định.

Link web MusicHUB: musichub-hcmut.vercel.app

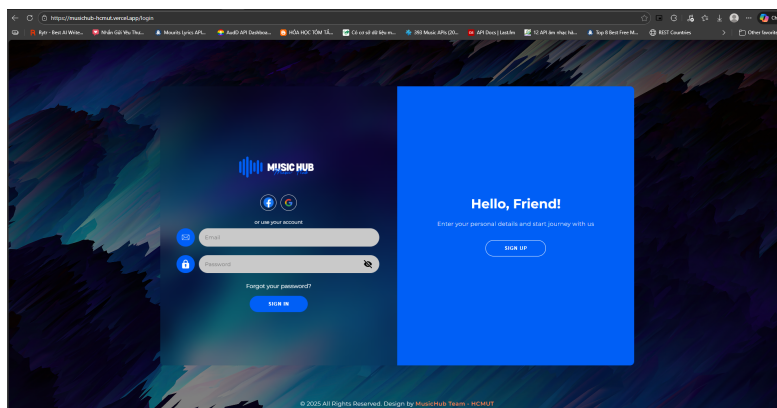
9.2 API Integration (Tích hợp API)

Hệ thống được xây dựng với định hướng tích hợp nhiều API từ các dịch vụ bên thứ ba nhằm mở rộng chức năng và giảm chi phí phát triển. Việc tích hợp API đóng vai trò trung tâm trong bảo mật, lưu trữ dữ liệu đa phương tiện và tương tác âm thanh. Nhờ đó, hệ thống có thể tận dụng các giải pháp có sẵn ngoài thị trường thay vì phải tự triển khai từ đầu.

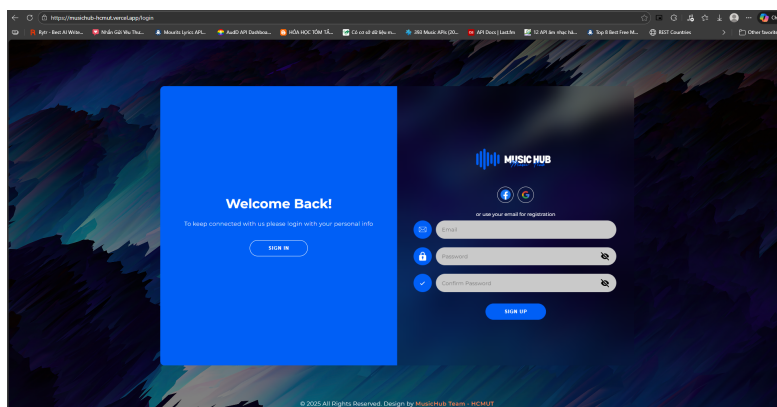
- **Cloud Platform Setup:** Ứng dụng sử dụng mô hình giao tiếp đa dịch vụ thông qua API, trong đó:
 - Người dùng có thể xác thực bằng Google thông qua OAuth 2.0, cho phép hệ thống lấy thông tin cơ bản (email, avatar) mà không yêu cầu tạo tài khoản thủ công.
 - hệ thống tích hợp Shazam API để nhận diện bài hát, backend gửi dữ liệu âm thanh được thu âm lên API và nhận về metadata phục vụ việc hiển thị trong ứng dụng.
 - Hệ thống còn sử dụng Cloudinary API để lưu trữ và xử lý hình ảnh (avatar, ảnh cover bài hát) thông qua cơ chế upload trực tiếp, giúp backend không phải quản lý file storage cục bộ.
- **RESTful API Design:** Ứng dụng lựa chọn RESTful API cho backend vì phù hợp mô hình CRUD và dễ triển khai với Node.js. Các endpoint được thiết kế theo nguyên tắc resource-based URL, ví dụ `/api/auth/google` cho xác thực Google. Tất cả tuân thủ chuẩn HTTP Methods như `GET` để truy vấn dữ liệu bài hát, `POST` để upload file và thực hiện đăng nhập, cùng với `DELETE` nếu cần xóa tài nguyên. Backend hoạt động stateless, tách biệt client-server và đảm bảo giao tiếp thống nhất qua JSON.
- **Testing and Validation:** Các API được kiểm thử bằng Postman để xác minh quy trình request-response, mô phỏng tình huống truy cập hợp lệ, sai token hoặc thiếu param. Các lỗi và mã trạng thái HTTP được xử lý theo tiêu chuẩn, ví dụ `400 Bad Request` khi payload upload Cloudinary không đúng, và `500 Internal Server Error` cho sự cố kết nối Shazam. Ngoài test thủ công, backend bổ sung unit test cho các hàm upload và xác thực, đồng thời thực hiện integration test với môi trường staging để đảm bảo giao tiếp ổn định giữa backend, Cloudinary và Shazam API trước khi triển khai hệ thống.

10 System Implementation

10.1 Login Page



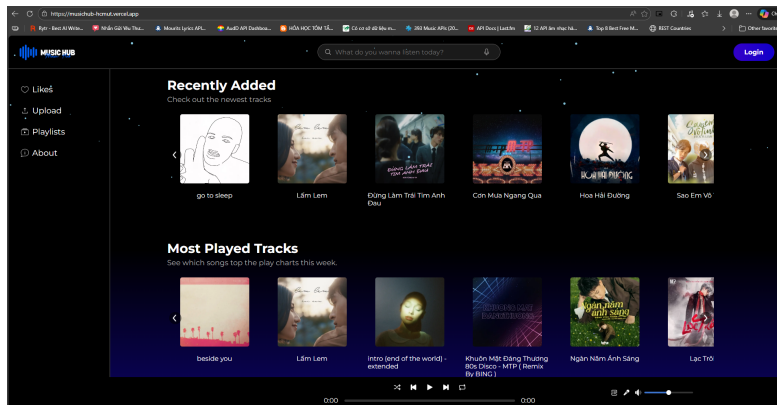
Hình 29: Giao diện trang đăng nhập



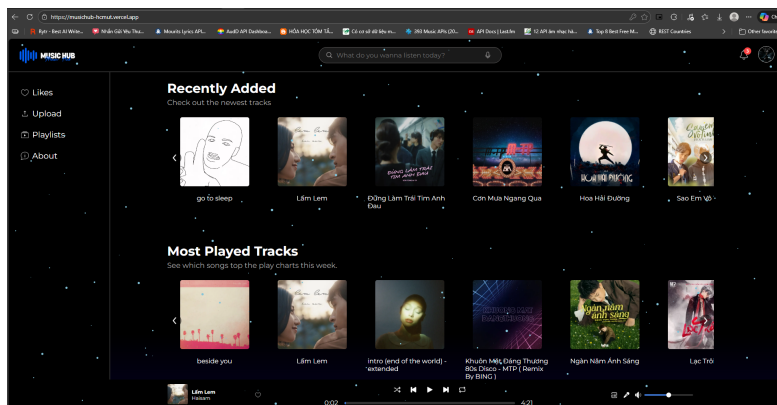
Hình 30: Giao diện trang đăng ký



10.2 Home Page

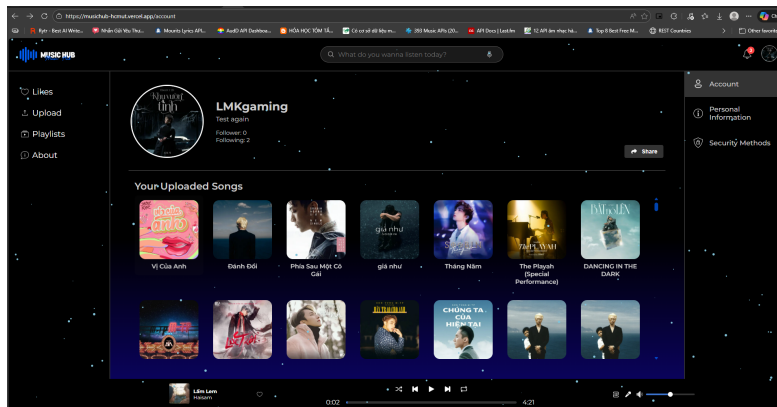


Hình 31: Giao diện trang chủ chưa đăng nhập

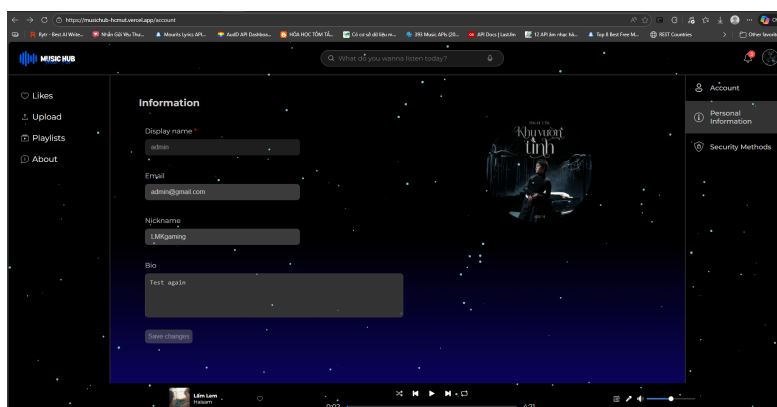


Hình 32: Giao diện trang chủ đã đăng nhập

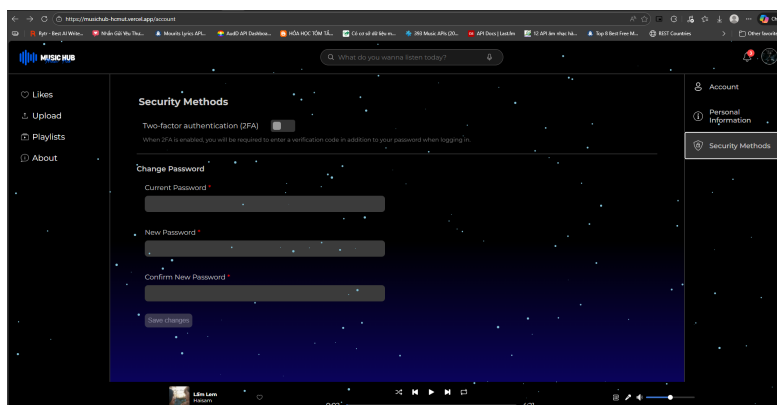
10.3 Account Page



Hình 33: Giao diện trang thông tin cá nhân

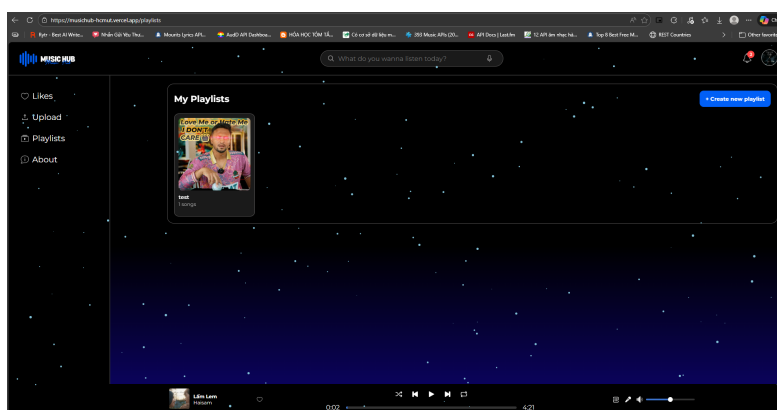


Hình 34: Giao diện trang chỉnh sửa thông tin cá nhân

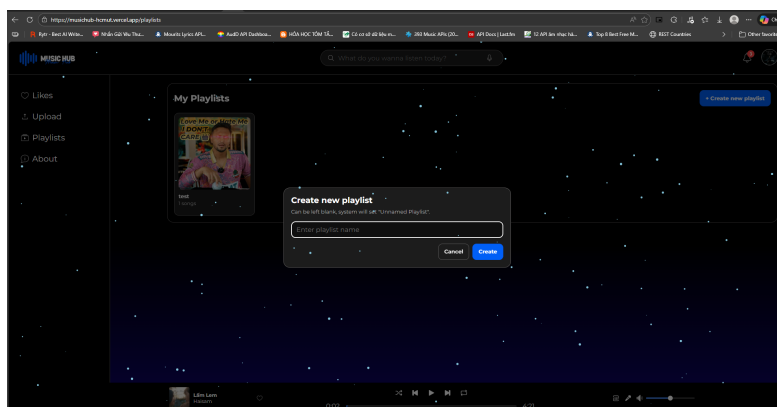


Hình 35: Giao diện trang quyền riêng tư và bảo mật thông tin cá nhân

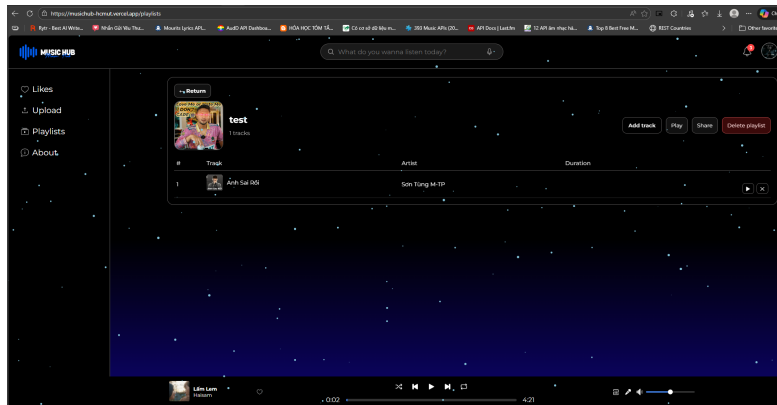
10.4 Playlist Page



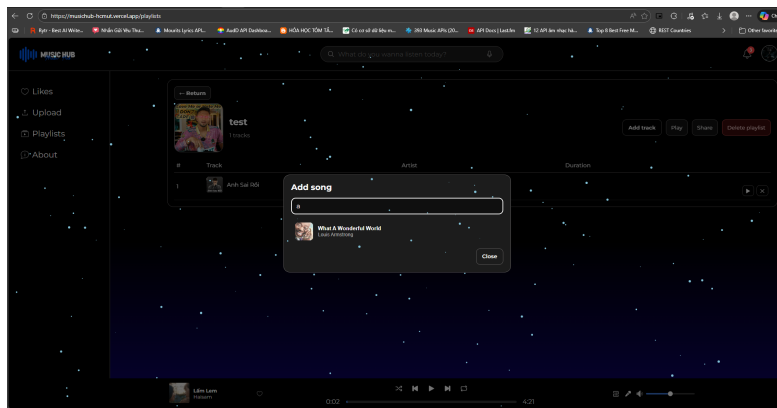
Hình 36: Giao diện danh sách playlists



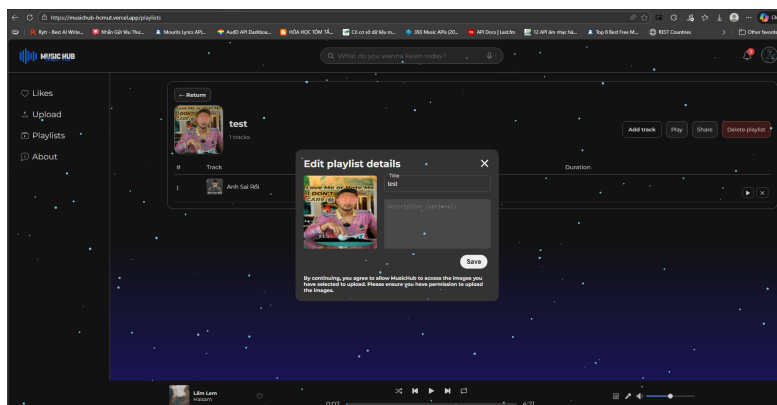
Hình 37: Giao diện tạo playlist



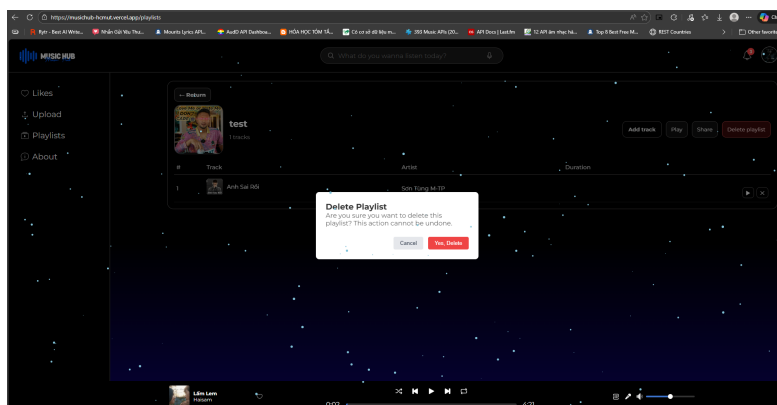
Hình 38: Giao diện danh sách bài hát của playlist



Hình 39: Giao diện trang thêm bài hát vào playlist

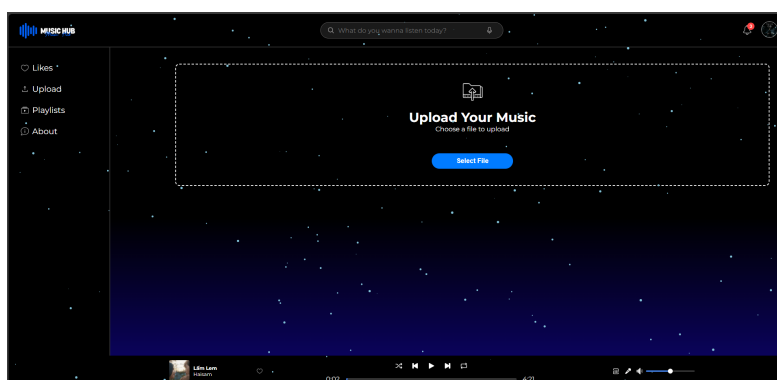


Hình 40: Giao diện chỉnh sửa thông tin playlist

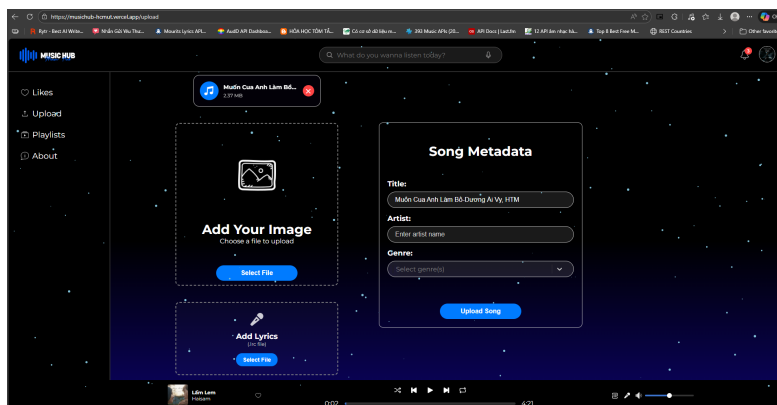


Hình 41: Giao diện xóa playlist

10.5 Upload Page

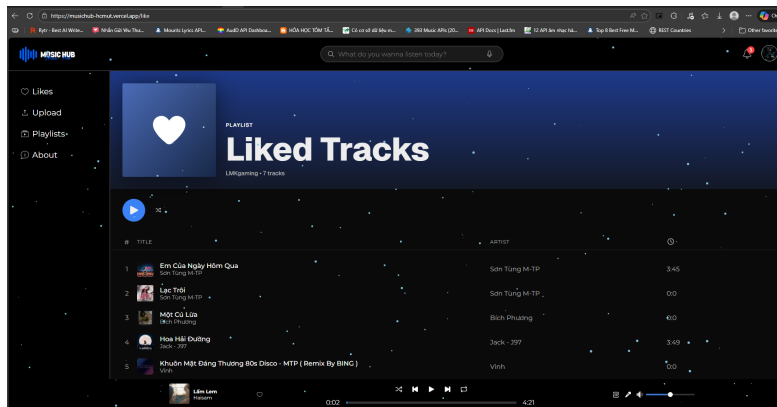


Hình 42: Giao diện upload bài hát cho artist



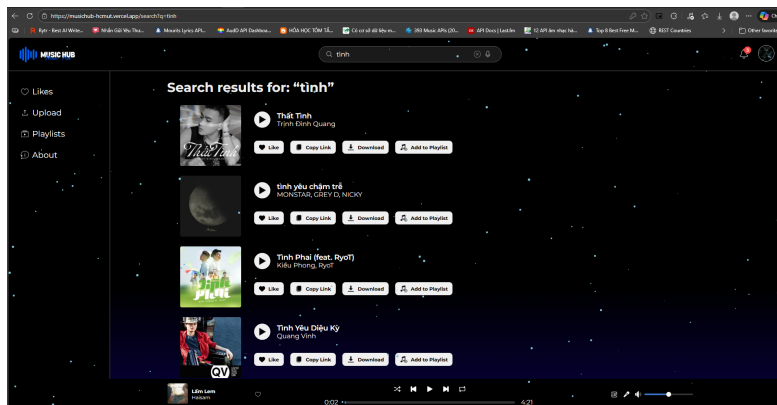
Hình 43: Giao diện chỉnh sửa thông tin bài hát

10.6 Liked Page



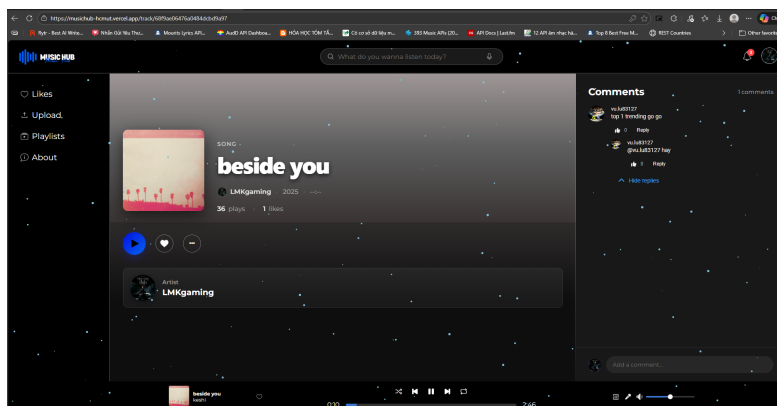
Hình 44: Giao diện danh sách bài hát yêu thích

10.7 Search Page



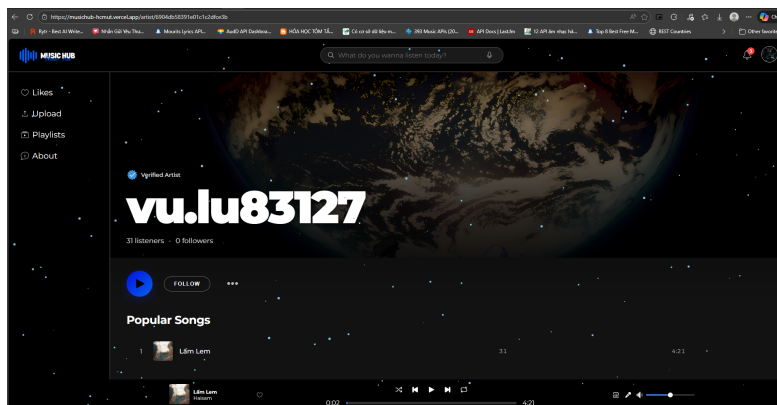
Hình 45: Giao diện kết quả tìm kiếm

10.8 Track Details Page



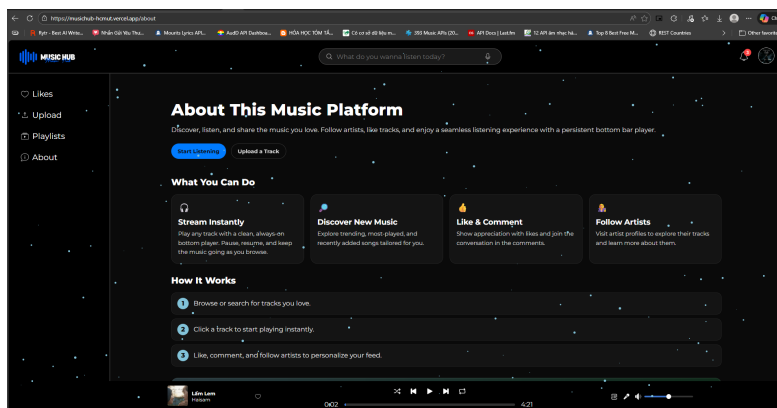
Hình 46: Giao diện thông tin và các bình luận của bài hát

10.9 Artist Details Page



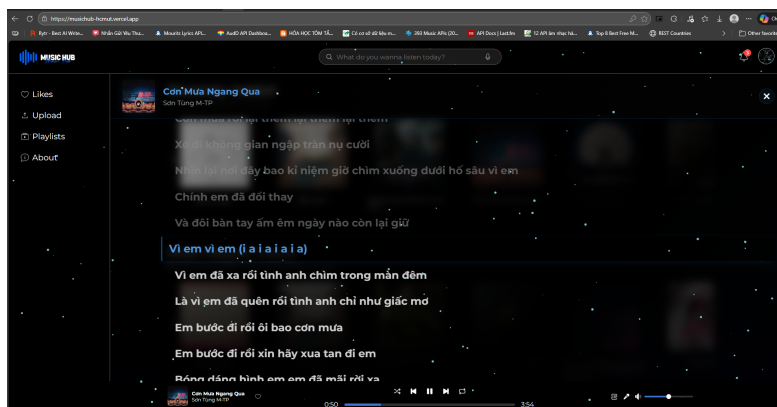
Hình 47: Giao diện thông tin và danh sách bài hát của artist

10.10 About Page



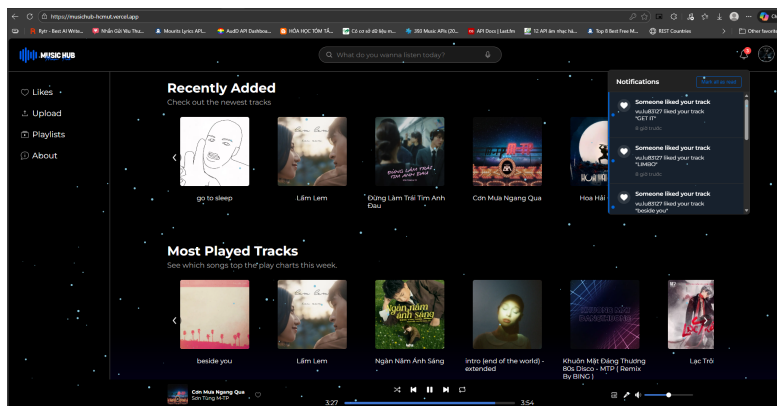
Hình 48: Giao diện trang thông tin ứng dụng

10.11 Lyrics Section



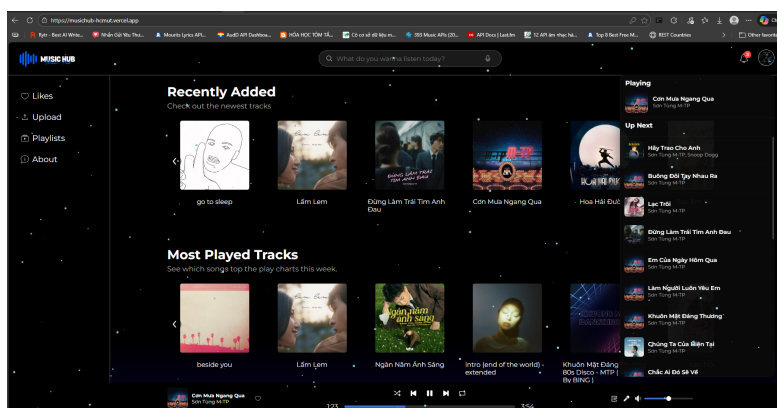
Hình 49: Giao diện hiển thị lời bài hát (đồng bộ với timeline)

10.12 Notification Section



Hình 50: Giao diện hiển thị danh sách thông báo của người dùng

10.13 Tracks Queue Section



Hình 51: Giao diện danh sách bài hát trong hàng đợi

11 Nhận xét, đánh giá và hướng phát triển

11.1 Kết quả đạt được

Sau quá trình nghiên cứu, thiết kế và hiện thực hóa, nhóm phát triển đã xây dựng thành công hệ thống MusicHUB với các kết quả chính như sau:

- Về mặt chức năng:
 - Xây dựng hoàn thiện luồng nghiệp vụ cốt lõi: Đăng ký/Dăng nhập, Upload nhạc, Quản lý Playlist và Phát nhạc trực tuyến.
 - Hiện thực hóa thành công tính năng **Adaptive Bitrate Streaming**, cho phép hệ thống tự động chuyển đổi giữa các mức chất lượng (64kbps, 128kbps, 320kbps) tùy thuộc vào băng thông mạng, đảm bảo trải nghiệm nghe nhạc không bị gián đoạn.

- Tích hợp các tính năng nâng cao mang lại trải nghiệm thú vị cho người dùng như: Đồng bộ lời bài hát (Lyric Sync) và Nhận diện bài hát qua giai điệu (Audio Fingerprinting) với độ chính xác khá tốt.
- Hệ thống gợi ý bài hát bước đầu hoạt động hiệu quả dựa trên lịch sử nghe nhạc gần đây của người dùng.
- **Về mặt giao diện và trải nghiệm người dùng (UI/UX):**
 - Giao diện được thiết kế hiện đại, trực quan, giúp người dùng mới dễ dàng làm quen trong thời gian ngắn.
 - Hỗ trợ tốt trên đa nền tảng (Web và Mobile Web), đáp ứng nhu cầu sử dụng linh hoạt trên nhiều thiết bị.
- **Về mặt kỹ thuật:**
 - Hệ thống đáp ứng được các yêu cầu khắt khe về hiệu năng, với độ trễ khi bắt đầu phát nhạc thấp ($\leq 800\text{ms}$) và khả năng xử lý đồng thời nhiều yêu cầu.

11.2 Hạn chế

Bên cạnh những kết quả đạt được, hệ thống vẫn còn tồn tại một số hạn chế cần khắc phục:

- **Thuật toán gợi ý:** Hiện tại thuật toán gợi ý chủ yếu dựa trên lịch sử nghe gần đây và lọc cộng tác cơ bản. Hệ thống gặp khó khăn với bài toán "Cold Start" (Khởi động lạnh) khi chưa có đủ dữ liệu hành vi của người dùng mới để đưa ra gợi ý chính xác.
- **Kho dữ liệu nhạc:** Do giới hạn về nguồn lực và bản quyền trong phạm vi đồ án, số lượng bài hát và metadata trong cơ sở dữ liệu còn hạn chế so với các nền tảng thương mại thực tế.
- **Tính năng nhận diện:** Chức năng nhận diện bài hát hoạt động tốt trong môi trường ít tạp âm, nhưng độ chính xác giảm xuống khi môi trường có nhiều tiếng ồn hoặc chất lượng thu âm từ thiết bị người dùng kém.
- **Khả năng mở rộng thời gian thực:** Chưa kiểm chứng được khả năng chịu tải thực tế với quy mô rất lớn (trên 10.000 người dùng đồng thời) trong môi trường production.

11.3 So sánh tính năng với các nền tảng âm nhạc hiện có

Các nền tảng nghe nhạc phổ biến hiện nay như Spotify và Apple Music tập trung mạnh vào kho nhạc bản quyền và đề xuất cá nhân hóa, nhưng không cho phép người dùng trực tiếp đăng tải nội dung và thiếu yếu tố tương tác cộng đồng theo từng bài hát. SoundCloud mở hơn cho nghệ sĩ độc lập nhưng chưa tích hợp khả năng nhận diện bài hát tự động, khiến người dùng phải sử dụng thêm ứng dụng khác như Shazam khi muốn xác định tên bài hát. Shazam lại chỉ giải quyết bài toán nhận diện, không hỗ trợ nghe, tải lên hay tạo không gian thảo luận. Từ những hạn chế đó, ứng dụng được đề xuất hướng đến việc kết hợp ba giá trị chính sau mà hiện chưa có sản phẩm nào đáp ứng đầy đủ:

- **Nền tảng mở:** bất kỳ người dùng nào cũng có thể đăng tải sản phẩm âm nhạc.
- **Nhận diện âm thanh tích hợp:** người dùng nhận diện một bài hát và nghe ngay trong cùng hệ thống.
- **Tương tác cộng đồng:** người dùng có thể bình luận, thảo luận từng bài hát — tạo hệ sinh thái xã hội cho âm nhạc.

11.4 Hướng phát triển trong tương lai

Để MusicHUB có thể cạnh tranh với các nền tảng thực tế và đáp ứng nhu cầu ngày càng cao của người dùng, nhóm phát triển đề xuất lộ trình nâng cấp hệ thống theo các hướng sau:

11.4.1 Về mặt Công nghệ và Kỹ thuật

1. **Nâng cấp hệ thống gợi ý với Deep Learning:** Thay thế các thuật toán lọc cộng tác cơ bản bằng các mô hình học sâu (Deep Learning). Hệ thống sẽ phân tích không chỉ lịch sử nghe mà còn cả ngữ cảnh thời gian thực (thời gian trong ngày, thời tiết, vị trí địa lý) để đưa ra gợi ý chính xác nhất về "tâm trạng" (mood) của người dùng.
2. **Phát triển Ứng dụng gốc (Native Mobile App):** Hiện tại hệ thống hoạt động trên nền tảng Web/Mobile Web. Hướng phát triển tiếp theo là xây dựng ứng dụng Native (sử dụng React Native hoặc Flutter) để tận dụng tối đa phần cứng thiết bị, cho phép thực hiện các tính năng quan trọng như:
 - **Chế độ Offline:** Tải nhạc về bộ nhớ thiết bị để nghe khi không có Internet.
 - **Background Processing:** Tối ưu hóa việc phát nhạc khi tắt màn hình và điều khiển qua tai nghe bluetooth tốt hơn.

11.4.2 Về mặt Tính năng và Nội dung

1. **Mở rộng loại hình nội dung (Podcast & Audiobooks):** Tận dụng hạ tầng streaming hiện có để hỗ trợ thêm các định dạng âm thanh lời nói (Spoken word) như Podcast và Sách nói, biến MusicHUB thành một nền tảng âm thanh tổng hợp.
2. **Tính năng Karaoke và Chấm điểm:** Dựa trên tính năng Lyric Sync đã có, hệ thống có thể phát triển thêm chế độ Karaoke, tách giọng hát ca sĩ (Vocal Remover bằng AI) và chấm điểm giọng hát người dùng, tăng tính giải trí và tương tác.
3. **Phòng nghe nhạc chung (Social Listening Party):** Tạo môi trường tương tác thời gian thực, cho phép người dùng tạo "phòng ảo", mời bạn bè cùng nghe một playlist đồng bộ, chat voice/text và thả cảm xúc (reaction) ngay trên giao diện phát nhạc.

11.4.3 Về mặt Thương mại và Quản lý

1. **Mô hình kiếm tiền (Monetization):**
 - Tích hợp quảng cáo âm thanh (Audio Ads) cho tài khoản miễn phí.
 - Triển khai gói đăng ký Premium (thanh toán qua Ví điện tử/Visa) để loại bỏ quảng cáo và mở khóa chất lượng nhạc Lossless.
2. **Cổng thông tin dành cho Nghệ sĩ (MusicHUB for Artists):** Cung cấp dashboard chuyên sâu hơn cho nghệ sĩ, bao gồm phân tích nhân khẩu học của người nghe, bản đồ phân bố fan hâm mộ, và công cụ quảng bá bài hát mới trực tiếp đến đối tượng mục tiêu.
3. **Ứng dụng Blockchain trong quản lý bản quyền:** Nghiên cứu tích hợp Smart Contracts để minh bạch hóa việc chi trả tiền tác quyền (Royalties) cho nghệ sĩ dựa trên số lượt nghe thực tế, đảm bảo quyền lợi công bằng cho người sáng tạo nội dung.